

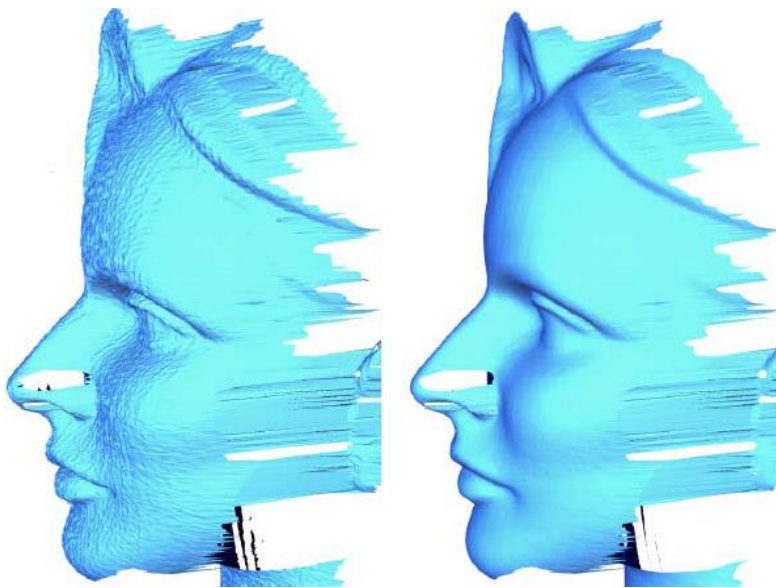
网格光顺、简化、细分与检索

高林

中国科学院计算技术研究所

网格光滑

- 去噪的目的：
 - ◆ 滤除高频噪音



网格光滑



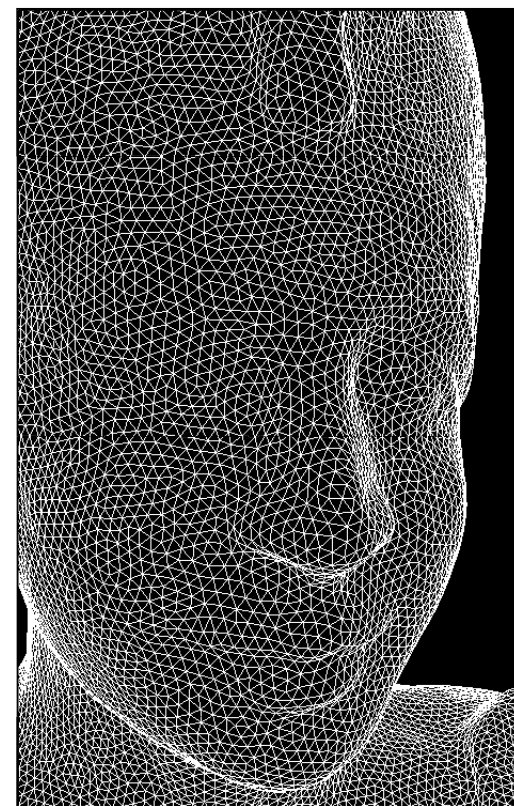
扫描
(Scan)



重建
(Reconstruct)



光滑
(Clean)



重网格化
(Remesh)

...

网格光滑

■ 难点：

- ◆ 去噪问题本质上是求解一个病态的逆问题：在噪声的类型和程度未知、真实模型的几何特性未知的前提下，如果要把噪声从输入中完美剥离，必然需要引入各种假设来辅助求解。
- ◆ 很多去噪算法对真实物体的几何特性做了很强的假设，比如曲面是 C^2 光滑，或者曲面类似CAD模型具有分片光滑和含有大量平面的特性。然而这一类的曲面只占三维物体类型中一小部分。

网格光滑

■ 难点：

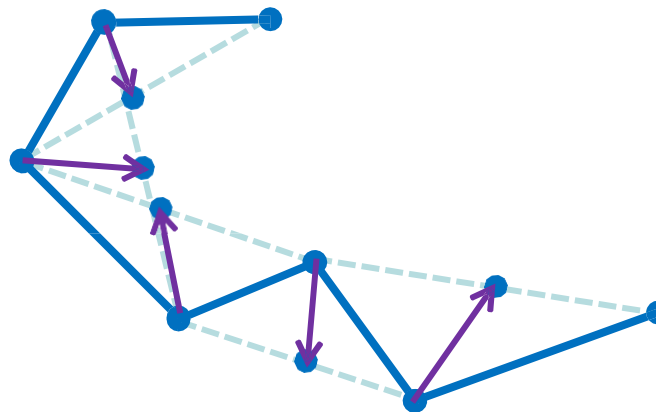
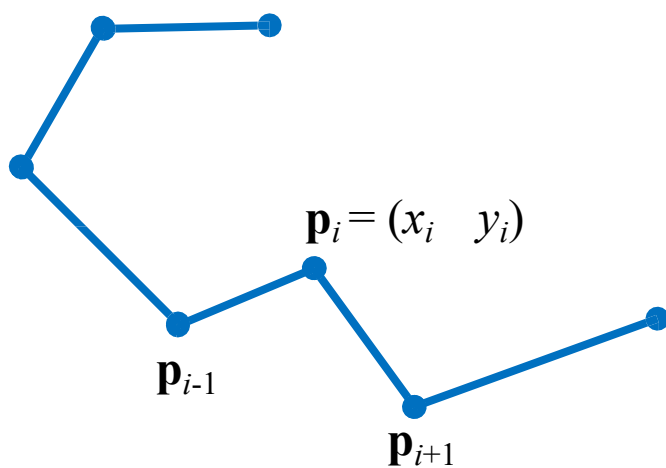
- ◆ 众多算法显式或隐含地对噪声模型做了假设，这些模型基本以高斯噪声为主，更广义一点，算法假设噪声是独立同分布。然而真实噪声的成因复杂，这些假设在很多情况下并不成立。针对高斯噪声的去噪方法在实际中也表现一般。

网格光滑

- 以三角网格曲面去噪为例，很多算法在已知网格上加若干噪声，然后用去噪结果和已知曲面的差异来比较算法的优劣。
- 两大缺陷：
 - ◆ 噪声类型人为给定，与真实不符。
 - ◆ 已知网格和加噪网格共享相同的网格连接关系，与真实情况不符。如无视这些缺陷，就无助于解决真实去噪问题。

Laplacian 曲线光滑

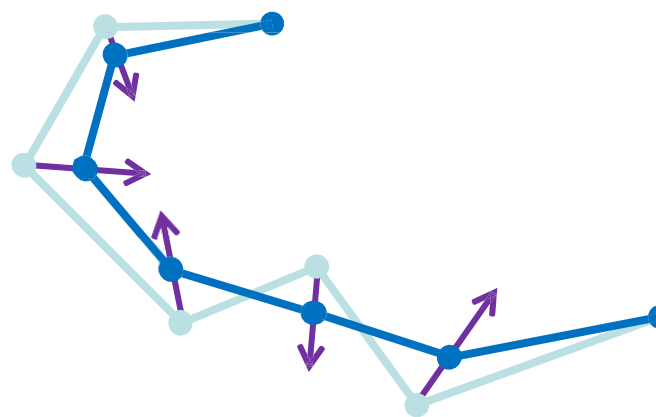
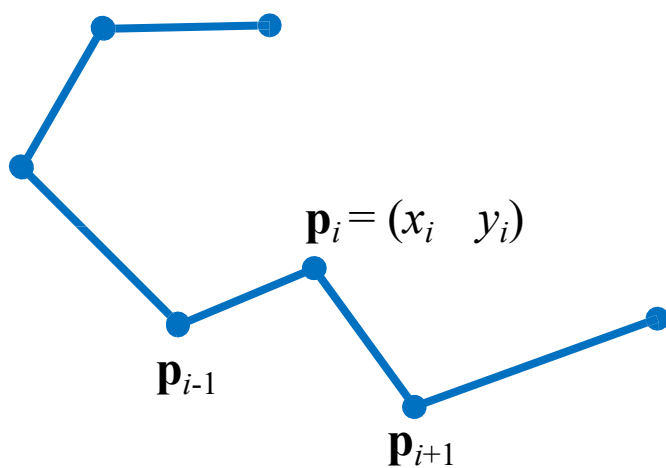
■ 曲面光滑



$$= \frac{1}{2}(p_{i+1} - p_i) + \frac{1}{2}(p_{i-1} - p_i)L(p_i) = (p_{i-1} + p_{(i+1)})/2 - p_i$$

Laplacian 曲线光滑

■ 曲面光滑



二阶导数的有限差分离散化
= 一维拉普拉斯算子



$$= \frac{1}{2}(p_{i+1} - p_i) + \frac{1}{2}(p_{i-1} - p_i)L(p_i) = (p_{i-1} + p_{i+1})/2 - p_i$$

Laplacian 曲线光滑

■ 算法:

- ◆ 迭代 m 次 (针对非边界点) (for non boundary points):

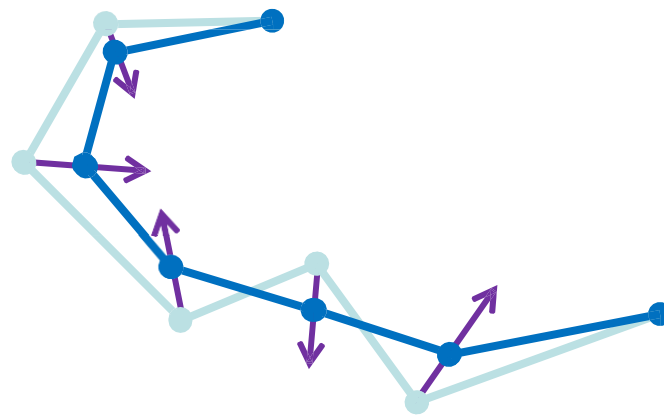
$$p_i \leftarrow p_i + \lambda L(p_i)$$

- ◆ λ 条件?

$$0 < \lambda < 1$$

- ◆ 近似曲线收敛?

- Single point

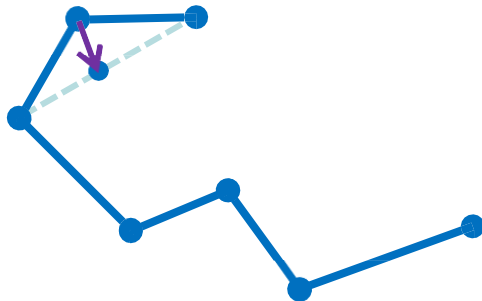


Laplacian 网格光滑

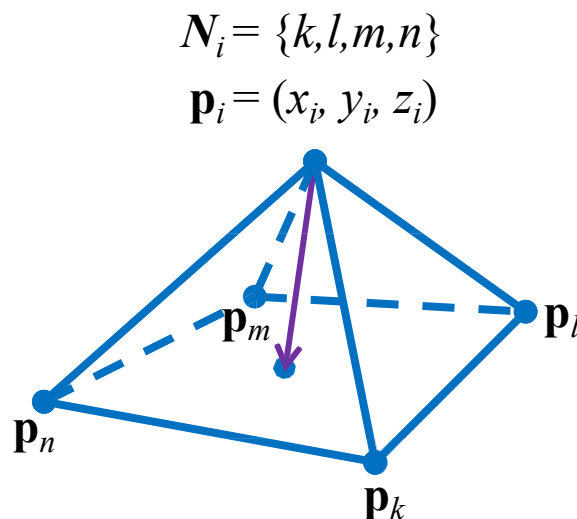
- 与曲面光滑类似:

$$\mathbf{p}_i^{(t+1)} \leftarrow \mathbf{p}_i^{(t)} + \lambda \Delta \mathbf{p}_i^{(t)}$$

- 三维网格中 $\Delta \mathbf{p}_i^{(t)}$ 如何定义?

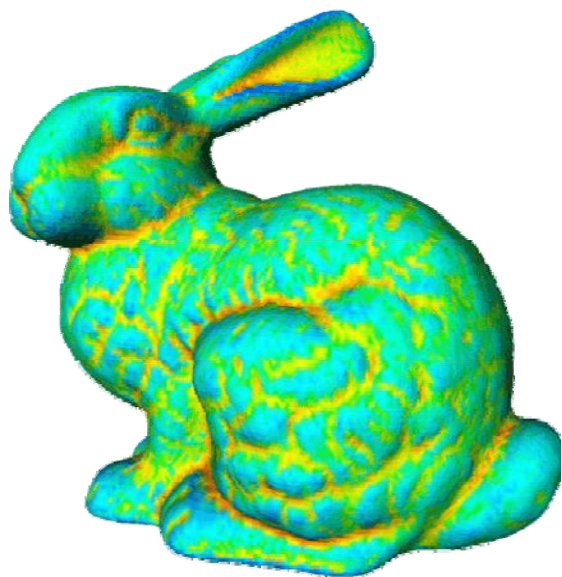


$$\frac{1}{2} (\mathbf{p}_{i+1} + \mathbf{p}_{i-1}) - \mathbf{p}_i$$

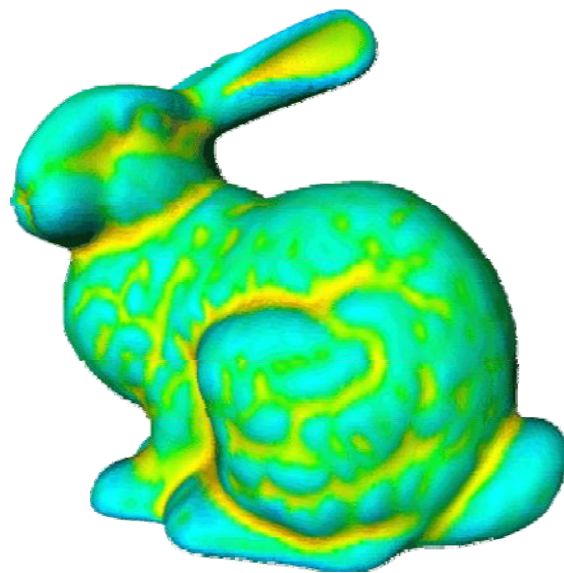


$$\frac{1}{|N_i|} \left(\sum_{j \in N_i} \mathbf{p}_j \right) - \mathbf{p}_i$$

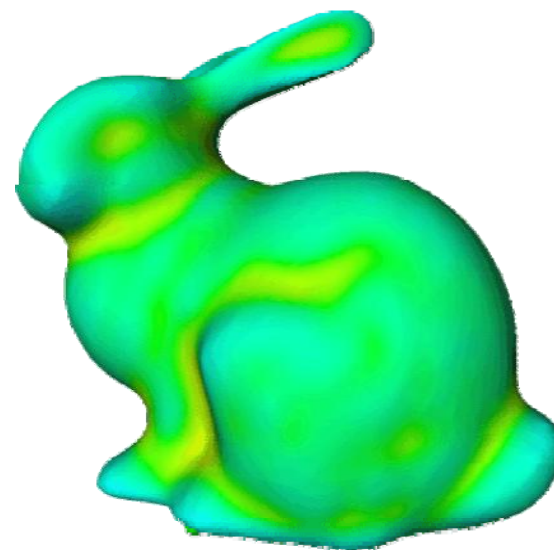
Laplacian 网格光滑



0 Iterations

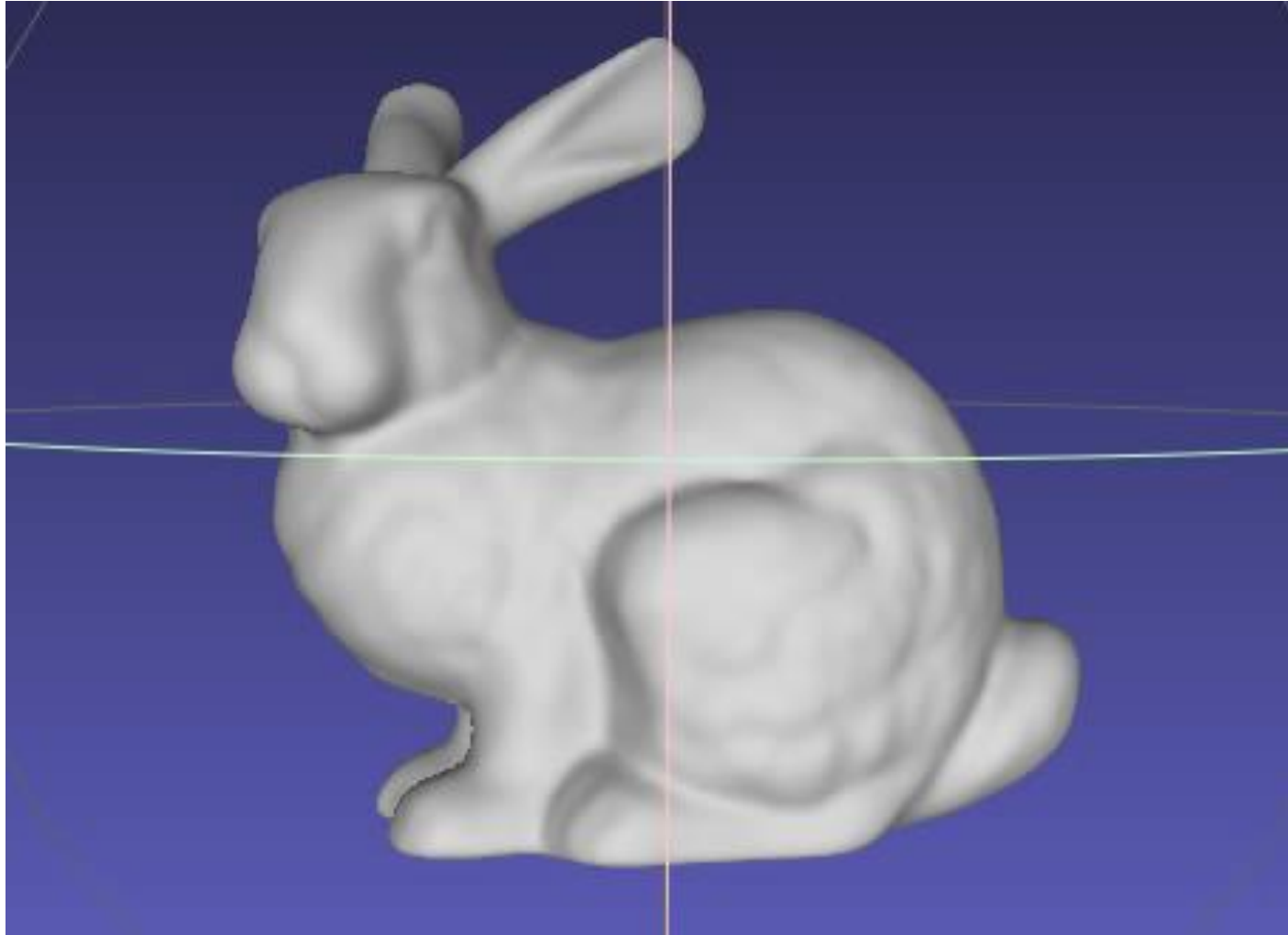


5 Iterations



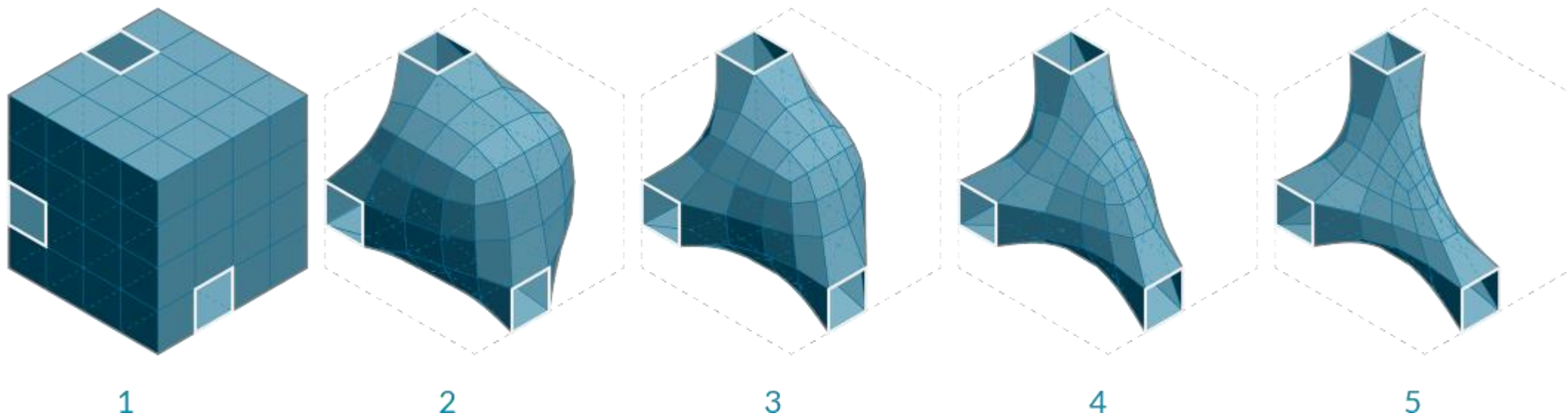
20 Iterations

Laplacian 网格光滑动画



Laplacian 网格光滑缺点

- 迭代次数过多，会导致网格变小



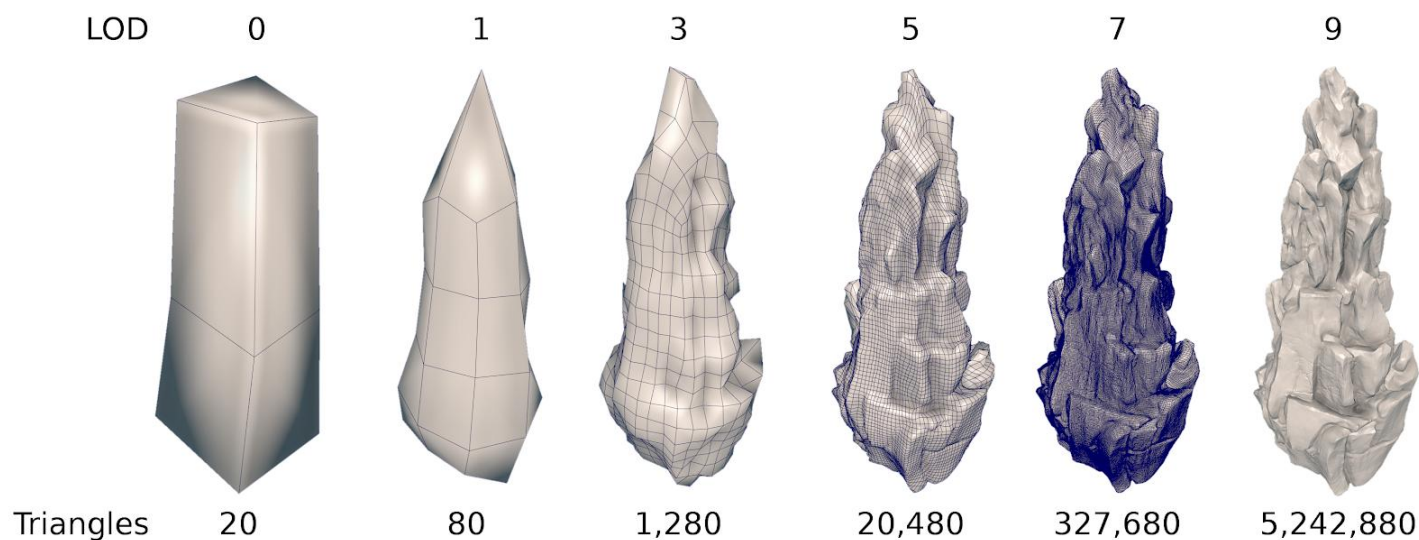
简化 (Simplification)

■ 网格简化的概念

- ◆ 用一些相对简单但维持几何特征的代表来近似一个给定的网格
- ◆ 移除几何冗余
 - 如一个有很多共面小三角形的平坦区域，将这些三角形融合成大的多边形能降低模型的复杂度，降低存储或传输的规模。
- ◆ 提高运行性能
 - 简化对于高效的渲染和编辑具有重要作用

简化 (Simplification)

- 产生场景中物体的层次细节 (level of details)。较远的物体用较低的层次细节呈现，较近的物体,用较高的层次细节呈现。
- 例子



简化 (Simplification)

■ 拓扑结构(Topology)

- ◆ 连通多边形网格的结构

- ◆ 亏格(genus)

 - 网格表面孔洞的数目。

 - 例如，一个球体或者立方体的亏格数目为零， 而一个环或者咖啡壶的亏格数目是一。

- ◆ 面片、边或顶点的局部拓扑特征邻域的连通性。

简化 (Simplification)

- 拓扑结构(Topology)

- 二维流形 (manifold)

- ◆ 如果局部拓扑处处等价于一个圆盘。 在三角网格流形拓扑中, 恰好仅有两个三角形 完全共边, 每个三角形分别与三个相邻的三角形有一条共边。

- 带边界的二维流形

- ◆ 确保边界, 使其仅属于一个三角形。

简化 (Simplification)

■ 技术方法: 如何简化?

几乎每一个简化技术都使用下面四种基本多边形移除技巧的组合或者变形:

- ◆ 采样 (sampling)
- ◆ 自适应细分 (adaptive subdivision)
- ◆ 去除 (decimation)
- ◆ 顶点合并 (vertex merging)

简化 (Simplification)

■ 采样(Sampling)

- ◆ 采样算法通过选取模型表面的点简单地对模型进行几何取样. 这些方法通常是复杂的并且难于编程实现。
- ◆ 这些方法难于精确地获取高频特征，通常在没有尖角的光滑表面上取得最好的效果。

简化 (Simplification)

■ 自适应细分 (adaptive subdivision)

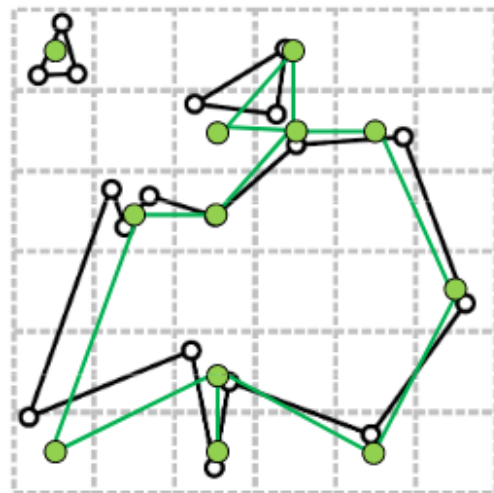
- ◆ 自适应细分算法通过寻找一个可以递归细分的基底网格来近似最初的模型。该算法在基底模型易于获取的情况下能取得最好效果。

- 例如，地形模型是一个典型四边形的情形。

- ◆ 自适应细分算法能够保持表面拓扑细节，因此有可能会降低对于大规模简化的处理能力。

■ 例子

- ◆ 一个简单的例子是将空间分割为一系列长方体子空间，
- ◆ 对子空间中的模型重新采样、简化为一个点得到简化结果



简化 (Simplification)

■ 去除(Decimation)

- ◆ 去除方法迭代地移除网格上的顶点或面片，并通过三角化填补每步移除后留下的孔洞。
- ◆ 这类方法相对简单、易于编程实现并且运行效率高。大部分的应用限制局部变化并且易于保持亏格，这类算法尤其精于处理移除几何冗余，比如共面多边形

简化 (Simplification)

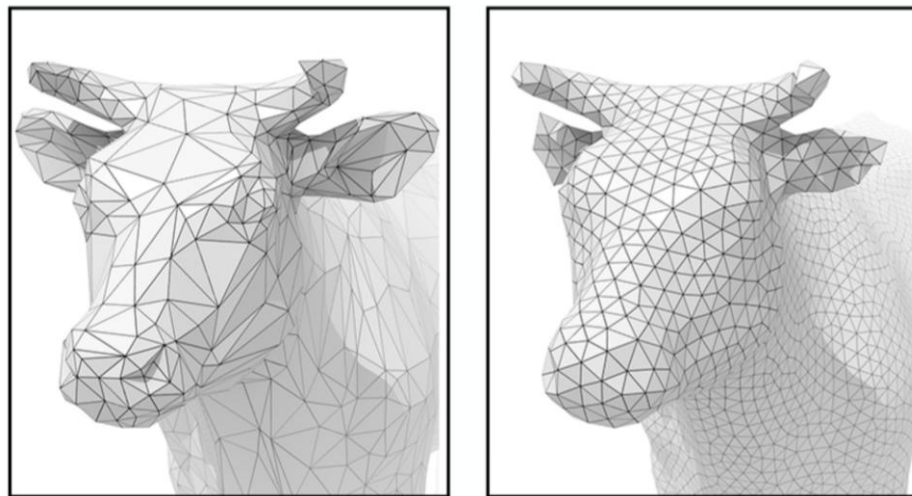
■ 顶点合并(Vertex-merging)

- ◆ 顶点合并方案通过将断裂三角化模型的两个或更多顶点合并为一个顶点，转而可以使之与其他顶点合并。该算法需要采用多种技术来决定以何种顺序合并顶点。
- ◆ 边坍塌算法(Edge-collapse algorithms), 一种总是将共边的顶点合并的算法, 易于保持局部拓扑, 但是允许一般顶点合并操作的算法应能够修改拓扑并且聚合物体。

网格细分

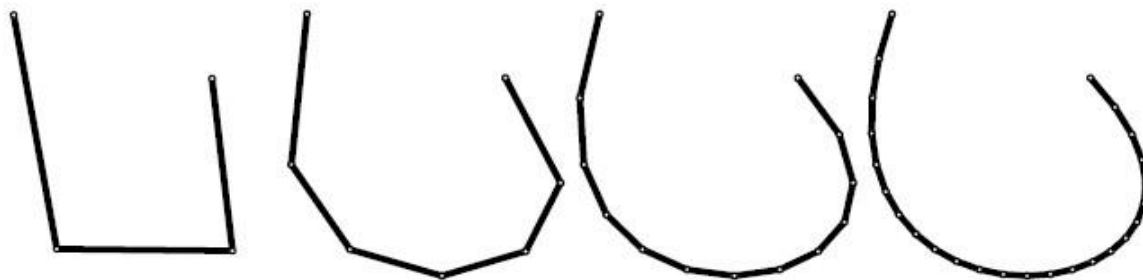
■ 细分(Subdivision)

- ◆ Catmull and Clark, Doo and Sabin在1978发表的论文标志着模型表面细分开始, 现在细分已大量应用于电影作品中。
- ◆ 细分的简略描述: 对于一个给定的原始网格, 进行精细地改进产生更光滑的效果



网格细分

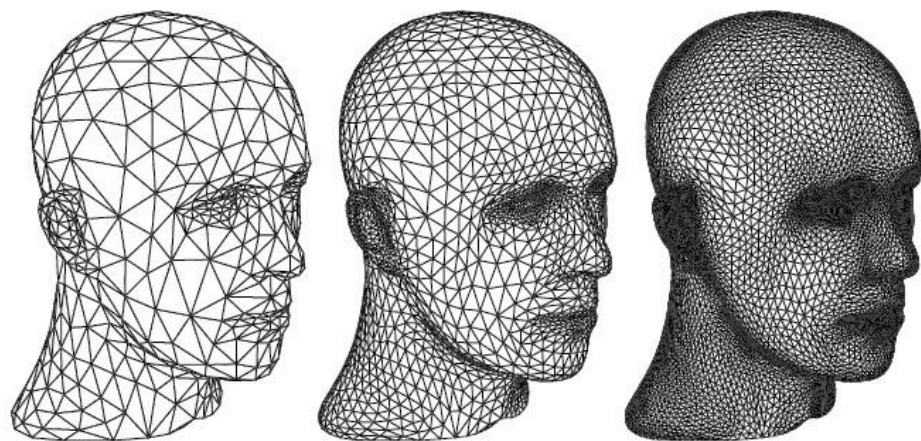
■ 一维细分的例子



- 最左面的4个点有平直线段相连，右边一个改进版本: 3个新的点被插入，经过进一步的两轮细分，曲线已经变得相当光滑。

网格细分

■ 细分的其他例子



- 三个连续的细分精炼结果. 最左边是最初的网格。根据特定的规则，每一个三角形被分割成四个小的三角形 (中间)。再次进行前面的分割得到最右边的结果。

网格细分

■ 细分(Subdivision)

- ◆ 表面细分可以被看成一个两阶段过程 (最初的网格被称作控制网格)。
 - 第一步, 称作细化阶段, 创建新顶点并与先前顶点相连产生新的、更小的三角形
 - 第二步, 称作平滑阶段, 计算新顶点的位置。
- ◆ 这两步的细节决定了不同的细分方案。在第一步中, 一个三角形可以不同的形式进行分割; 在第二步中, 新顶点的位置可以不同的方式插值产生。

网格细分

- 我们将介绍两个细分方案

Loop细分

$\sqrt{3}$ 细分

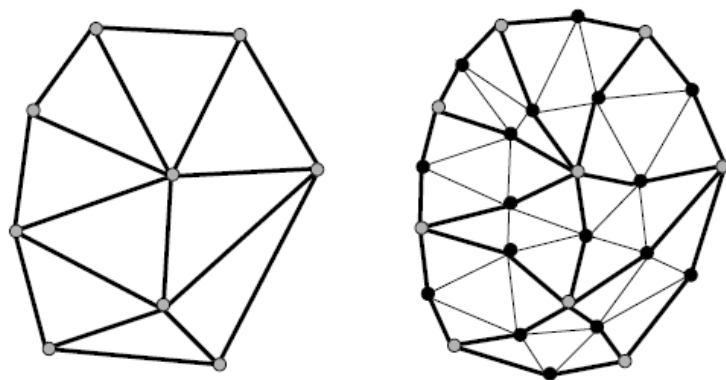
网格细分

■ Loop细分

- ◆ Loop 细分是第一个基于三角网格的细分方案，它更新每个已有的顶点并对每条边创建一个新的顶点，然后每个三角形被分割成四个新的三角形。因此，经过 n 步细分，一个三角形被分割成 $4n$ 个三角形。

■ Loop细分

- ◆ Loop细分的例子。新的顶点以黑色显示，每条边中加入一个新的顶点，新的顶点被连接起来创建四个新的三角形以替换掉原先的三角网格。



网格细分

■ 细分准则的相关

- ◆ 关注一个已存在的顶点 p^k , 其中 k 是当前已进行的细分步数。按此记法, p^0 即表示控制网格的顶点。一般地, 细分过程可以表示为

$$p^0 \rightarrow p^1 \rightarrow p^2 \rightarrow p^3 \dots$$

- ◆ 如果 p^k 的度为 n , 即 p^k 有 n 个相邻顶点, 记做

$$p_i^k, i \in \{0, 1, 2, \dots, n-1\}$$

网格细分

■ 细分规则(Subdivision rule)

- ◆ 接下来给出Loop细分方案的规则。

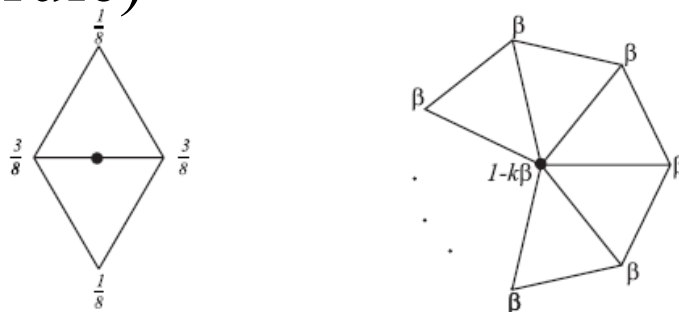
$$p^{k+1} = (1 - n\beta)p^k + \beta(p_0^k + \cdots + p_{n-1}^k)$$

$$p_i^{k+1} = \frac{3p^k + 3p_i^k + p_{i-1}^k + p_{i+1}^k}{8}, i = 0, \cdots, n - 1$$

- ◆ 第一个公式是将每个顶点 p^k 升级为 p^{k+1} 的准则;
- ◆ 第二个公式在边 $p^k p_i^k$ 创建新顶点 p_i^{k+1} 的准则。

网格细分

■ 细分规则(Subdivision rule)



这张图给出了插入顶点的规则模板(左边是在边上加入一个新顶点, 右边是对顶点的更新调整)

$$\beta \text{ 是 } n \text{ 的函数 } \frac{1}{n} \left(5/8 - \left(\frac{3}{8} + \frac{1}{4} \cos \frac{2\pi}{n} \right)^2 \right)$$

β 的选择需要保证细分无穷次的极限在度为6的顶点附近形成 C^2 曲面, 不为6的附近形成 C^1 曲面。

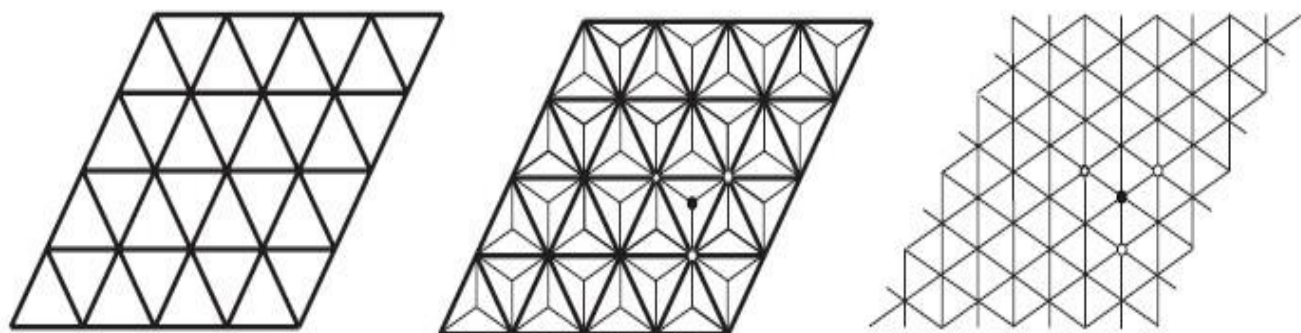
网格细分

■ $\sqrt{3}$ 细分

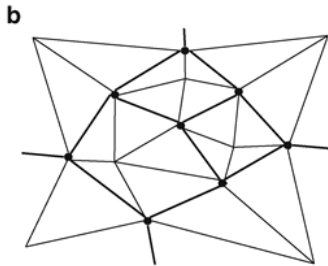
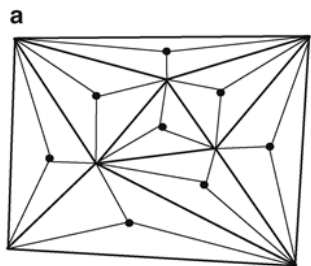
- ◆ Loop细分将每个三角形分割成4个新的小三角形, 因此 以 $4mn$ 的比率创建新的三角形, 其中 m 是控制网格中的三角形, n 是细分的步数。
- ◆ 不同于Loop细分, Kobbelt的 $\sqrt{3}$ 方案每一步只创建三个新的三角形。它在每个三角形中间创建一个新的顶点 (称作中间顶点), 而不是在每个边上创建一个顶点。

网格细分

为了得到更均匀分布的三角形, 每一个原始三角形都被翻转以使之连接两个相邻的中间顶点而不是连接两个已经存在的顶点。

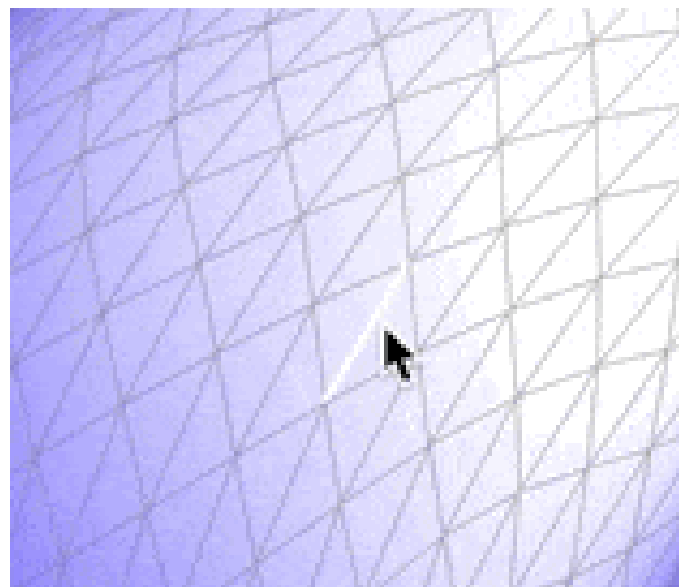


$\sqrt{3}$ 细分的过程



— Old edges
— New edges
• New vertices

边翻转



网格细分

■ 细分规则(Subdivision rule)

- ◆ 在下面第一行公式中, p_m 表示中间顶点, 由计算三角形三个顶点 p_a, p_b, p_c 的平均值得来。
- ◆ 对于每一个存在的顶点, p^k 用第二行公式来做更新, 其中 p_i^k 表示 p^k 的相邻顶点。在Loop方案中, n 是 p^k 的度, k 是细分的步数。

$$p_m^{k+1} = (p_a^k + p_b^k + p_c^k)/3$$

$$p^{k+1} = (1 - n\beta)p^k + \beta \sum_{i=0}^{n-1} p_i^k$$

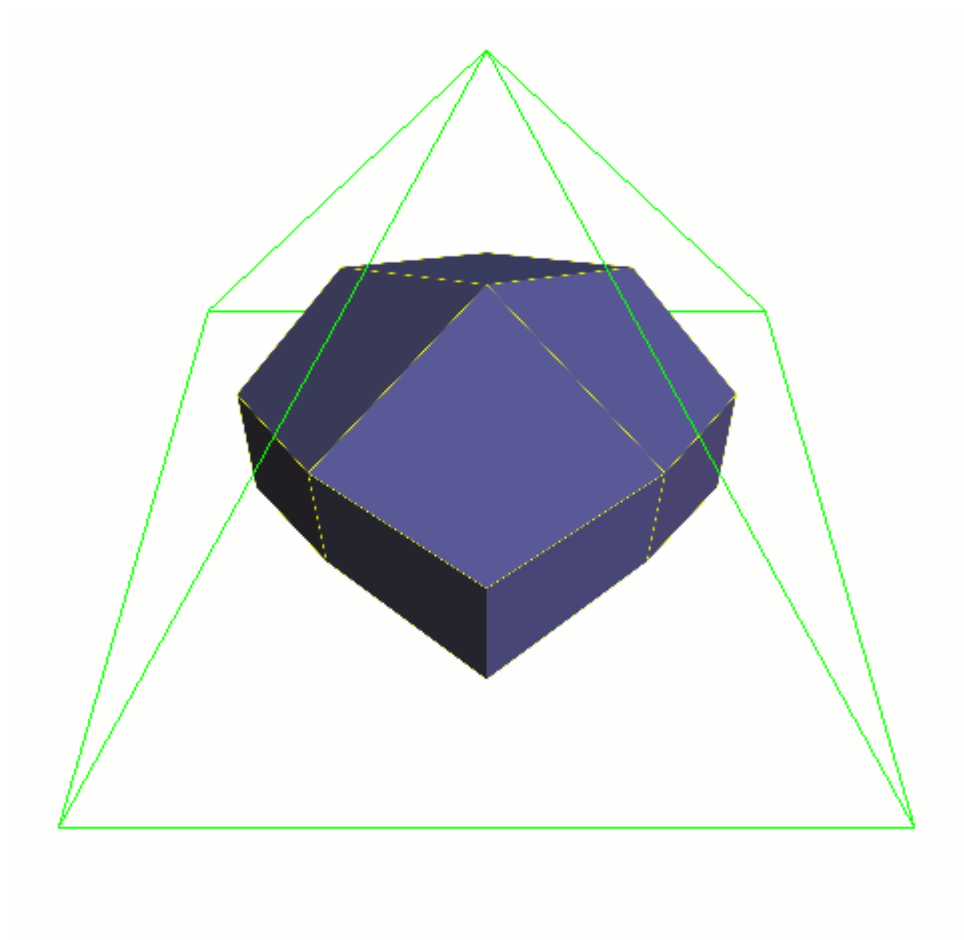
网格细分

■ 细分规则(Subdivision rule)

- ◆ 进一步, β 是一个度 n 的函数, 并且下面的 $\beta(n)$ 保证了无限细分的极限结果能够在顶点附近得到 C^2 连续的曲面。

$$\beta(n) = \frac{4 - 2\cos(2\pi/n)}{9n}$$

网格细分示例



Leif Kobbelt

- 1994年卡尔斯鲁厄理工学院博士学位
- 2000年获得海因茨-迈尔-莱布尼茨奖
- 2008年提名为欧洲图形学协会Fellow
- 2013年亚琛工业大学杰出教授
- 开源软件OpenMesh创始人



Charles Loop

- 1992年华盛顿大学博士
- 2009年被奥斯卡授予技术成就奖
- Nvidia首席研究科学家
- Loop细分算法发明人，该算法被收录入了Pixar公司的OpenSubDiv软件包，广泛用于电影产业。



网格模型简化

- 真实感的**实时绘制**问题变的日益突出，因为做实时绘制，而开始了模型简化。
- 92年开始，面片简化、压缩、信号处理、纹理合成成为SIGGRAPH会议的一个热门话题；如今网格模型的处理，如拓扑修复、细节建模、模型的动画变形仍然是研究热点；从而演化为数字几何处理这一研究方向。
- Eurographics Symposium on Geometry Processing

网格模型简化

- 层次细节简化技术：在不影响画面视觉效果的前提下，通过逐次简化景物的表面细节来减少场景的几何复杂性，提高绘制算法的效率。
- 层次细节简化技术主要应用在：
- 简化采样密集的多面体网格激光扫描测距系统，扫描真实三维物体三维场景的存储/传输/及绘制

网格模型简化

- 该技术对一个原始多面体模型建立几个不同逼近程度的几何模型。
- 从**近处**观察物体时，采用精细的模型，从**远处**观察物体时，采用较粗糙的模型。当**视点连续变化**时，在两个不同层次的模型之间存在一个明显的跳跃，有必要在相邻层次的模型之间形成光滑的视觉过渡，即几何形状过渡 (geomorphs)。

网格模型简化

■ 总之，层次细节简化技术的研究主要集中在：

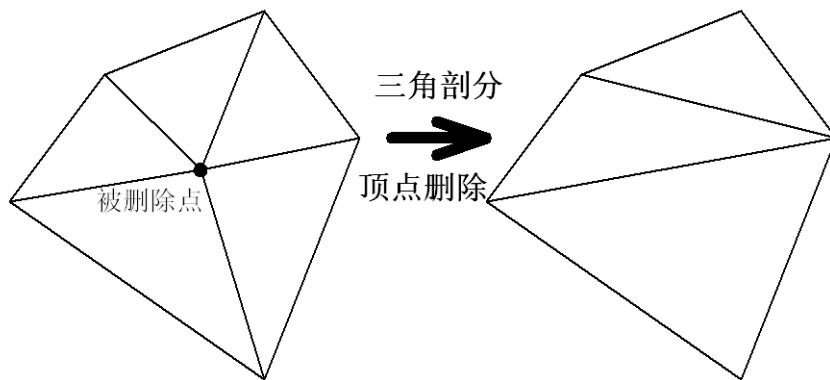
① 建立不同层次细节的模型。对任意给定的复杂多边形网格 M ，由精细至粗糙建立一模型序列：其中 $M = M_0$

② 建立相邻层次的多边形网格 M_i, M_{i+1} ($0 \leq i < n$) 之间的几何形状过渡： $\psi: M_i \rightarrow M_{i+1}$

网格简化

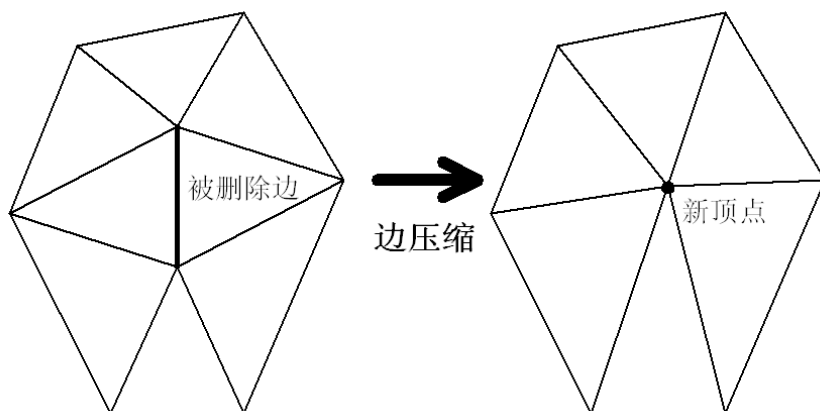
■ 三种不同的基本化简操作：

1. **顶点删除操作：** 删除网格中的一个顶点，然后对它的 相邻三角形形成的空洞作三角剖分



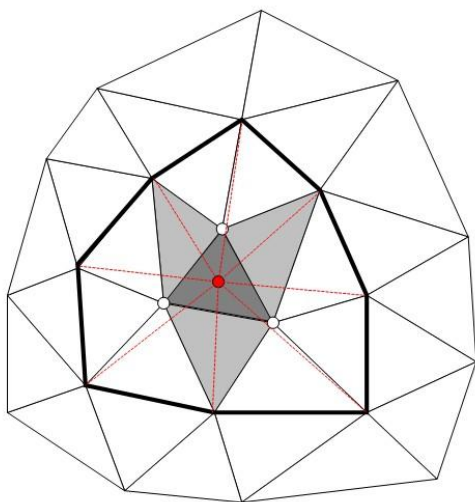
网格简化

2. **边压缩操作：** 网格上的一条边压缩为一个顶点，与该边相邻的两个三角形的退化



网格简化

3. **面片收缩操作：** 网格上的一个面片收缩为一个顶点， 该三角形本身和与其相邻的三个三角形都退化



网格模型简化

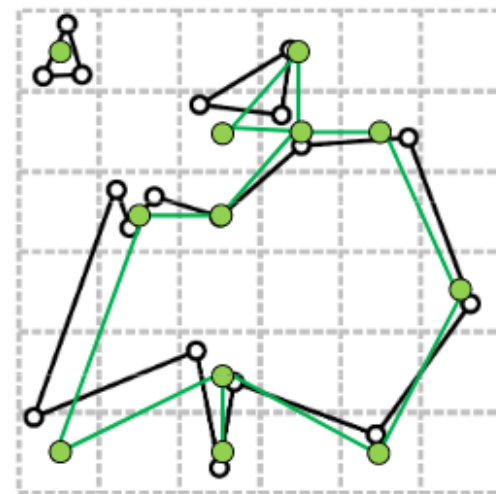
■ 基于长方体滤波的多面体简化

- ◆ 1993年, Rossignac和Borrel提出一个实用的、能实时建立LOD模型的多面体简化算法。

网格模型简化

■ 长方体滤波的步骤 (1)

- ◆ 给定一个多面体 M ，记 K 为其拓扑，假设 M 已三角化。算法首先建立 M 的长方体包围盒，并将该包围盒所包围的空间均匀剖分成一系列的小长方体子空间，然后采用各长方体子空间对景物顶点进行聚类合并，位于同一长方体空间的顶点被归于一类(Cluster)。



二维示例

网格模型简化

■ 长方体滤波的缺点

- ◆ 方法的主要缺点是顶点的合并导致了一些重要高频细节的丢失

参考文献

Rossignac J, Borrel P, Multi-Resolution 3D approximations for rendering complex scenes, in Modeling in Computer Graphics, edit by B Falcidieno and T L Kunii, spring-Verleg, 1993, pp 455-465.

网格模型简化

■ 顶点删除技术

■ 想法:

- ◆ 设法减少景物表面的采样点数目。假设景物表面已离散为一系列三角形，顶点删除算法首先从原始模型的顶点集中删除一些不重要的顶点，同时从其面片集中删除与这些顶点相连的所有面片。经上述操作后在原景物表面上留下了一些空洞，算法再对这些空洞进行局部三角剖分。

网格模型简化

■ 局部判别准则

- ◆ 基于相邻面片和边界的局部平坦性原则 Schroeder
- ◆ 采用等距面来限定简化模型顶点的变化范围 Cohen

网格模型简化

■ Schroeder的局部判别准则

◆ 该判别准则分两种情况：

1) 对网格内部顶点 v ，记其周围相邻面片集为 S ，则该点的平坦性标准由下述的距离来描述：

$$d = |N \cdot (v - C)|$$

其中 N 为向量 $\frac{\sum_{f \in S} n(f)A(f)}{\sum_{f \in S} A(f)}$ 的单位向量， $C = \frac{\sum_{f \in S} c(f)A(f)}{\sum_{f \in S} A(f)}$ ，这里的 $A(f)$ ， $c(f)$ ， $n(f)$ 分别为三角面片的面积、中心和法向量。

网格模型简化

■ Schroeder的局部判别准则

2) 对边界顶点 v , 记与它相邻的两个边界顶为 v_1 , v_2 , 则其平坦性标准定义为 v 到 v_1 与 v_2 连线的距离。

参考文献

Schroeder W, Zarge J A, and Lorensen W E, Decimation of triangle meshes, Computer Graphics, 1992, 26-2, 65-70.

网格模型简化

■ 渐进的网格简化技术

- ◆ 渐进网格算法中，任一网格 $\hat{M}$ 均可表示为一粗网格及 n 个逐步细化网格 M^i ($i = 1, 2, \dots, n$) 的变换，且有 $\hat{M} = M^n$
- ◆ 一张网格 M 可定义为 1 个二元组 (K, V) ，其中 K 是一个单纯复形(simplicial complex)，它表示了 M 的顶点、边和面的邻接关系；
- ◆ 单纯复形 K 由顶点集 $\{i = 1, 2, \dots, m\}$ 及其称之为单形的非空子集组成。0-单形 $\{i\} \in K$ 即为顶点，1-单形 $\{i, j\} \in K$ 为一条边，而2-单形 $\{i, j, k\} \in K$ 为一个面。

网格模型简化

■ 显示能量函数度量

- ◆ 有了上述定义，Hoppe采用显式能量函数 $E(M)$ 来度量 简化网格与原始网格的逼近度([HOPP96]):

$$E(M) = E_{dist}(M) + E_{spring}(M) + E_{scalar}(M) + E_{disc}(M) \quad (1)$$

- ◆ 其中, $E_{dist}(M)$ 为 M 的距离能量, 定义为点集 $x = \{x_1, \dots, x_n\}$ 到网格 M 的距离平方:

$$E_{dist}(M) = \sum_{i=1}^n d^2(x_i, \phi_V(|K|))$$

网格模型简化

■ 显示能量函数度量

- ◆ $E_{spring}(M)$ 为 M 的弹性能量，这相当于在 M 的每条边上均放置一条弹性系数为 k 的弹簧，即

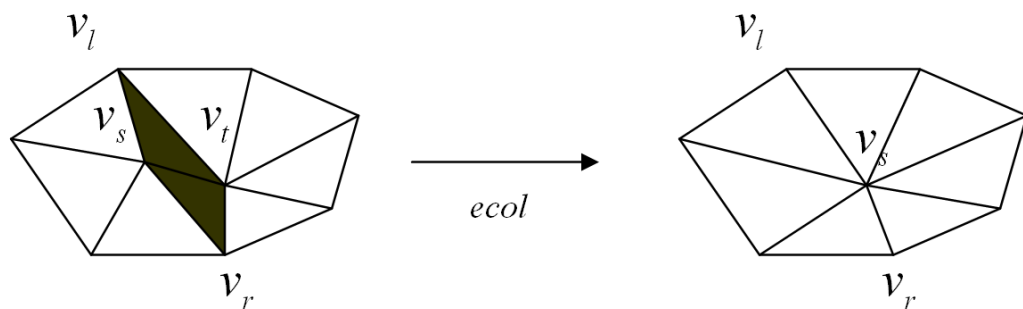
$$E_{spring}(M) = \sum_{(i,j) \in K} k \|v_i - v_j\|^2$$

- ◆ $E_{scalar}(M)$ 为度量 M 的标量属性的精度，而 $E_{disc}(M)$ 则度量了上视觉不连续的特征线（如边界线、侧影轮廓线等）的几何精度。

网格模型简化

■ 边收缩变换

- ◆ Hoppe利用边收缩变换(edge collapse transformation)来 逐步迭代计算上述能量的优化过程。
- ◆ 下图给出了一个边收缩变换的过程，它将该边的两个端点(v_s, v_t) 收缩为一个顶点 v_s 。经过这个变形后，其相邻两个面 $\{v_s, v_t, v_l\}$ 和 $\{v_s, v_t, v_r\}$ 均退化为一边。



网格模型简化

因此，初始网格 $\hat{M} = M^n$ 可经过 n 组的边收缩变形后简化为 M^0 ：

$$\hat{M} = M^n \xrightarrow{ecol_{n-1}} \dots \xrightarrow{ecol_1} M^1 \xrightarrow{ecol_0} M^0$$

网格模型简化

■ 顶点分裂变换

- ◆ 顶点分裂变换(vertex split transformation)
- ◆ 由于边收缩变换 *ecol* 是一可逆变换，我们称其逆变换为顶点分裂变换。Hoppe 利用式(1)来选择这些变换，使变换前后两网格间的能量差 ΔE 达到最小。

参考文献

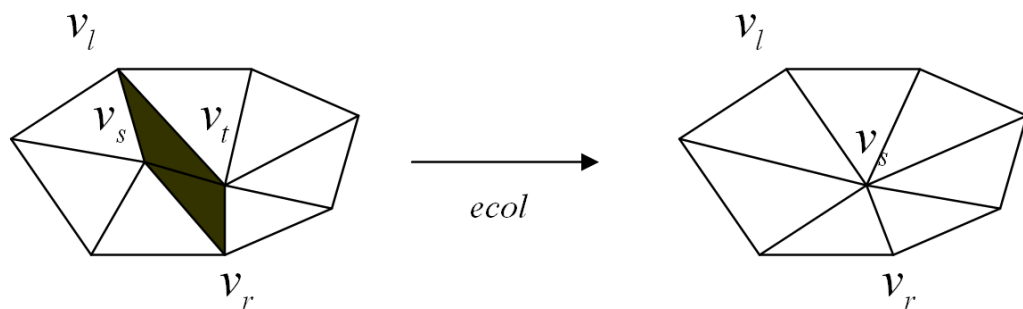
Hoppe H, DeRose T, Duchamp T, Mesh Optimization, Computer Graphics, 1993,27-4,19-26.

Hoppe H, Progressive Meshes, Computer Graphics, 1996, 30-4, 99-108.

网格模型简化

■ 基于二次误差度量的简化技术

- ◆ Hoppe的边收缩(edge collapse)操作可推广为一般的顶点合并变换来描述 $(v_1, v_2) \rightarrow v$ ，其含义是将场景中的两个顶点 v_1, v_2 移到一新的位置 v ，将连向 v_1, v_2 的所有边都连向 v ，并删除所有退化的边和面片。

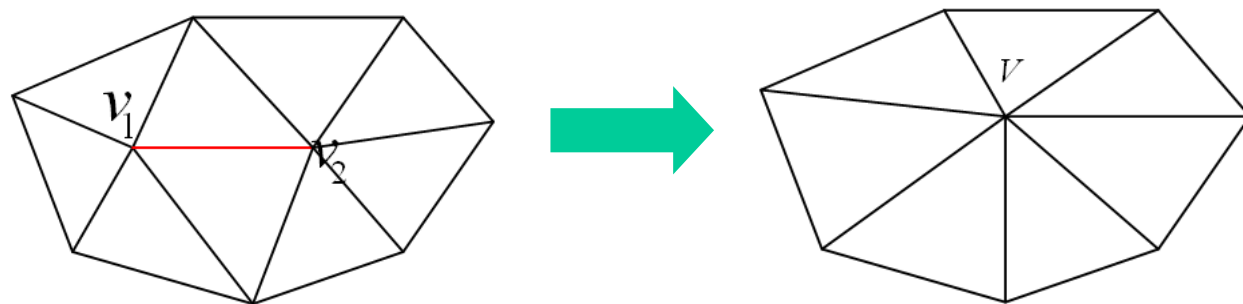


网格模型简化

■ 点对合并 (1)

◆ 点对 (v_1, v_2) 合并的原则

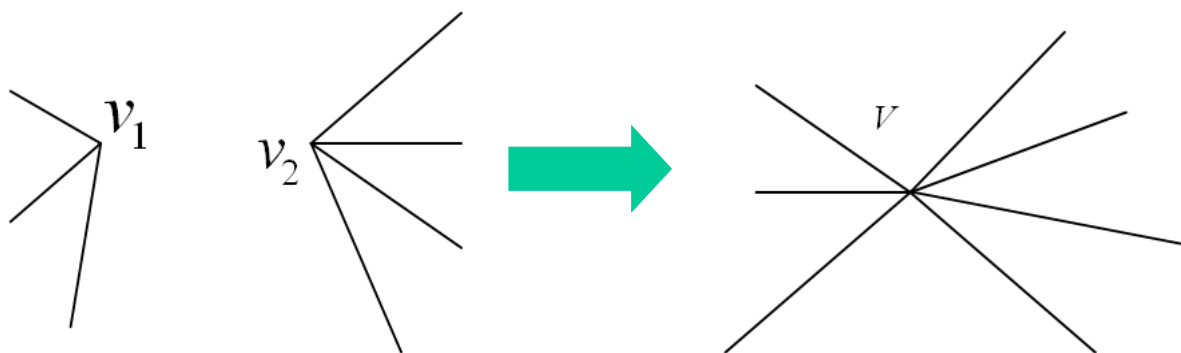
(1) v_1, v_2 为某一表面上的相邻点, 即 v_1, v_2 为一条边



网格模型简化

■ 点对合并 (2)

(2) $\|v_1 - v_2\| < t$, t 为用户给定的阈值参数



网格模型简化

■ 二次误差度量

- ◆ Garland和Heckbert引进了二次误差度量来刻画每一个顶点移动后引起的误差。对表面上的每一个顶点均有许多三角面片与之相邻，记 $plane(v_a)$ 为这些三角形所在的平面方程所构成的集合，即

$$plane(v_a) = \left\{ (a, b, c, d) \left| \begin{array}{l} ax + by + cz + d = 0, (a^2 + b^2 + c^2 = 1) \\ \text{is coefficients of the adjacent plane of } v_a \end{array} \right. \right\}$$

网格模型简化

则我们采用如下的二次函数来度量 v_a 移动到 v 时产生的误差：

$$\Delta(v_a \rightarrow v) = \sum_{p \in \text{plane}(v)} (pv^T)^2$$

其中 $v = (x, y, z, 1)$ 为齐次坐标。展开上式得到

$$\Delta(v_a \rightarrow v) = \sum_{p \in \text{plane}(v)} (pv^T)^2 = v \left(\sum_{p \in \text{plane}(v)} K_p \right) v^T = vQ(v_a)v^T \quad (2)$$

网格模型简化

(2)式中

$$K_p = \begin{bmatrix} a^2 & ab & ac & ad \\ ab & b^2 & bc & bd \\ ac & bc & c^2 & cd \\ ad & bd & cd & d^2 \end{bmatrix}$$

这样，对每一顶点 v_a ，在预处理时，我们均可按上述方法计算矩阵 $Q(v_a)$ ，进而就可对其移动进行误差度量了。但由于每次合并时，需同时移动两点，故必须考虑同时移动多个顶点后形成的误差。

网格模型简化

■ 多点移动的误差

- ◆ Garland和Heckbert简单地采用加法规则来刻画多点移动而形成的误差，对点对合并 $(v_1, v_2) \rightarrow v$ ，其误差为

$$\Delta(v) = \Delta(v_1 \rightarrow v) + \Delta(v_2 \rightarrow v) = v(Q(v_1) + Q(v_2))v^T = vQv^T$$

其中， $Q = Q(v_1) + Q(v_2)$ 。因而，应选取 v 使误差达到最小。

网格模型简化

由极值的性质知, v 满足系统方程:

$$\frac{\partial \Delta(v)}{\partial x} = \frac{\partial \Delta(v)}{\partial y} = \frac{\partial \Delta(v)}{\partial z} = 0$$

$$\text{即 } \begin{bmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} v = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

若上式有唯一解, 则其解为 v 的最优解; 否则, 用伪逆技术求 v 。若伪逆技术失败, 则简单地选取 v 为 v_1, v_2 或 $\frac{v_1+v_2}{2}$ 中的任何一个。

网格模型简化

参考文献

Garland M., Heckbert P S., Surface simplification using quadric error matrix, Computer Graphics, 1997, 209-216.

网格模型简化

■ 例子

1. Hoppe 的Progressive Meshes:

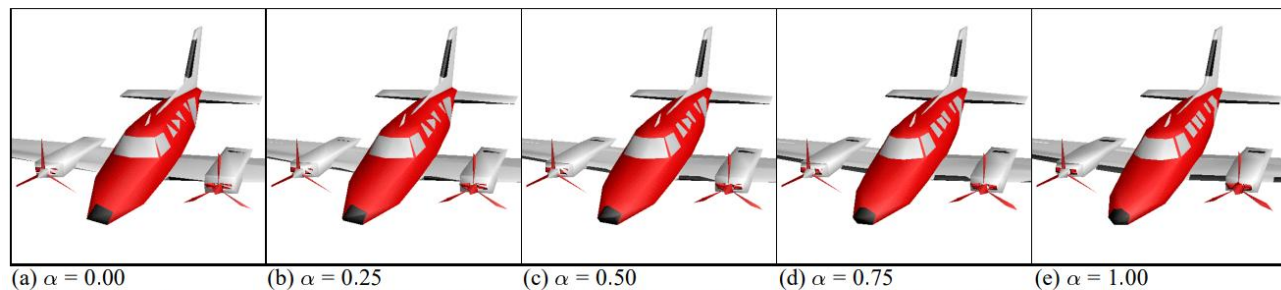
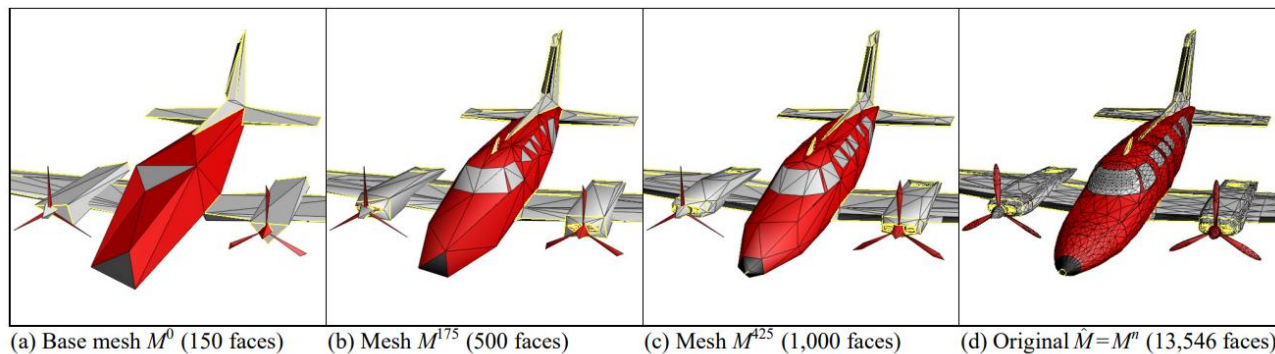


Figure 6: Example of a geomorph $M^G(\alpha)$ defined between $M^G(0) \doteq M^{175}$ (with 500 faces) and $M^G(1) = M^{425}$ (with 1,000 faces).

Hugues Hoppe

- 引用40666
- 1994年华盛顿大学博士
- 导师: Tony Deroose
- 成名作Progressive Meshes他引4864次
- 2004获ACM SIGGRAPH成就奖, 2011年获选ACM Fellow



网格模型简化

■ 例子

2. Garland的牛

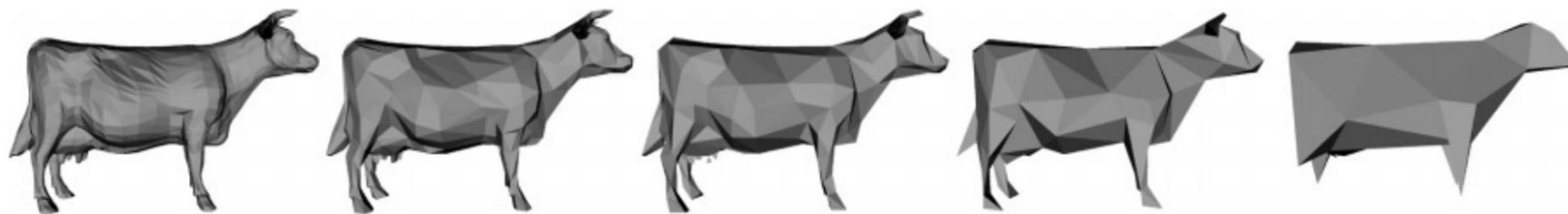


Figure 4: A sequence of approximations generated using our algorithm. The original model on the left has 5,804 faces. The approximations to the right have 994, 532, 248, and 64 faces respectively. Note that features such as horns and hooves continue to exist through many simplifications. Only at extremely low levels of detail do they begin to disappear.

三维形状描述

- 提出了许多**全局和局部**的三维形状描述符。参见[Tangelder & Veltkamp, A survey of content based 3D shape retrieval methods, 2008]



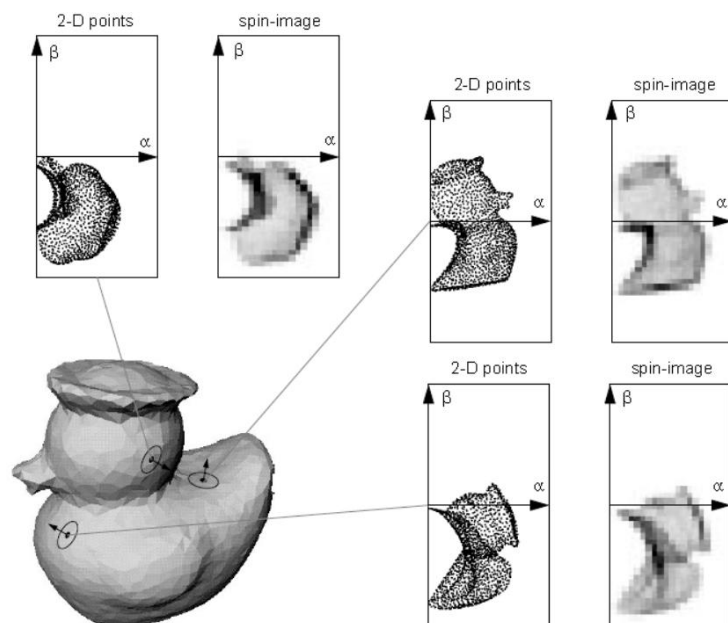
◆ 2D图像描述符的主要优势:

◆ **对外平面位姿变化的鲁棒性**

- 当前的方法要么是将3D图形的位姿对齐到标准坐标系(例如, 使用PCA), 要么是使用以对象为中心的表示

Spin Images

- 广泛使用的三维形状描述符
- 将三维点转换为二维图像，用于相似性比较
- **以对象为中心的表示法**，因此不随姿态变化

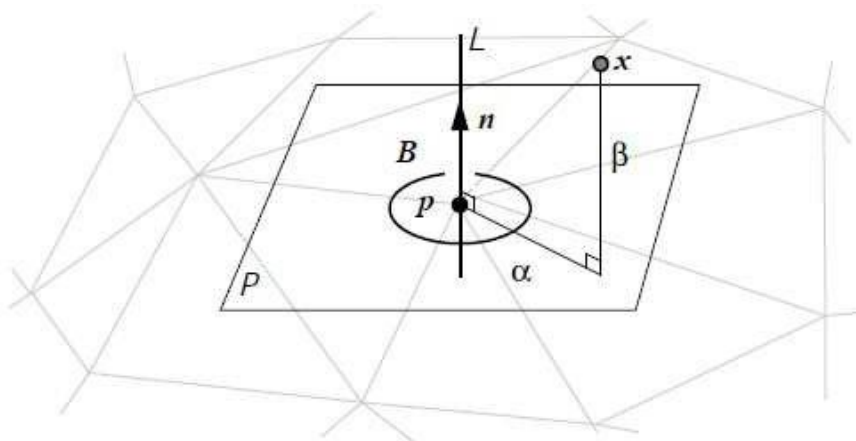


3D形状中的点被表示为图像(它们如何看到其他点)

Spin Images

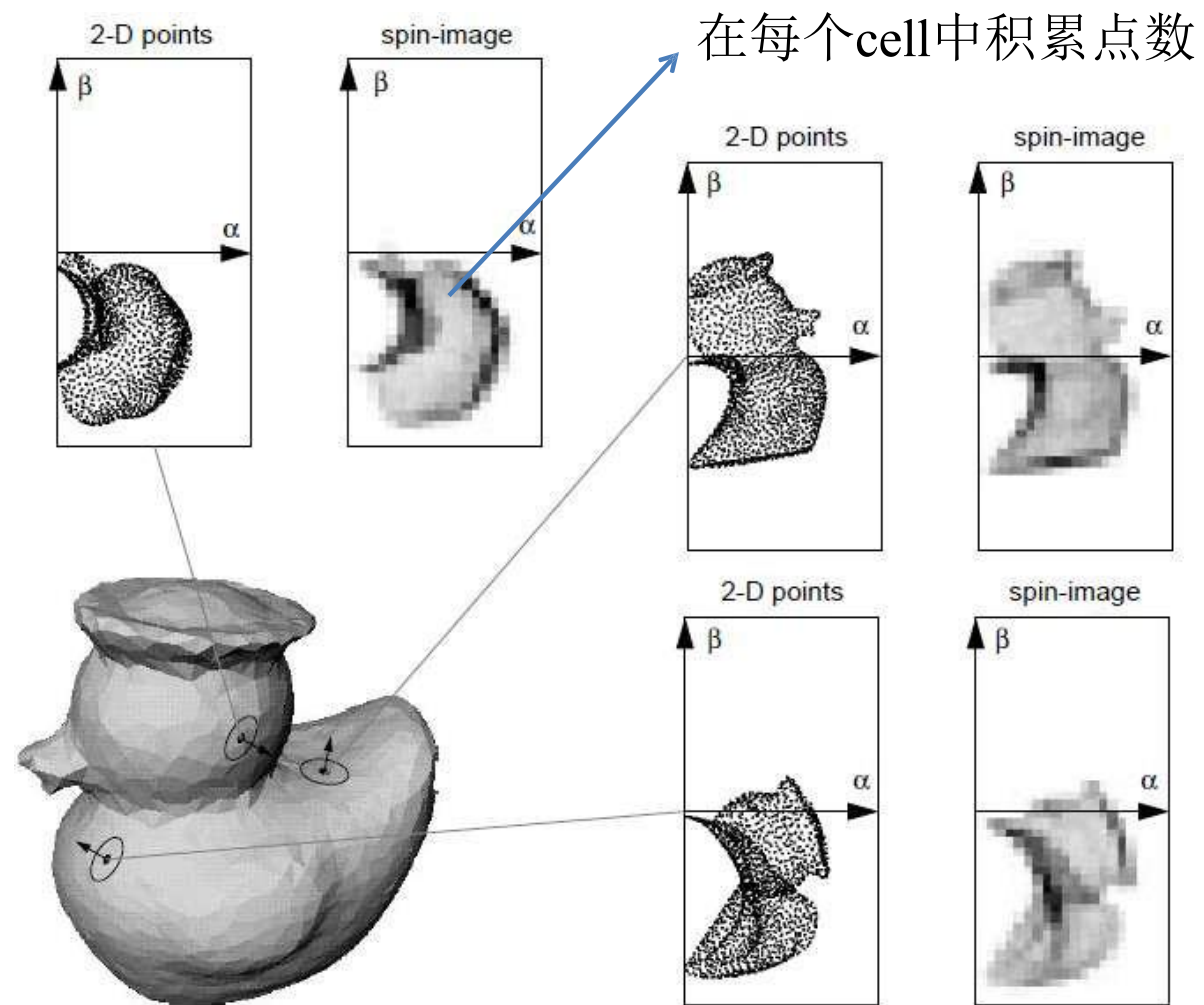
■ 我们如何形成一个基点 p 的像？

- ◆ 通过垂直于点 p 的表面法线的 p 方向来定义切平面
- ◆ 然后，通过计算径向距离和高程，将所有其他点映射到平面上



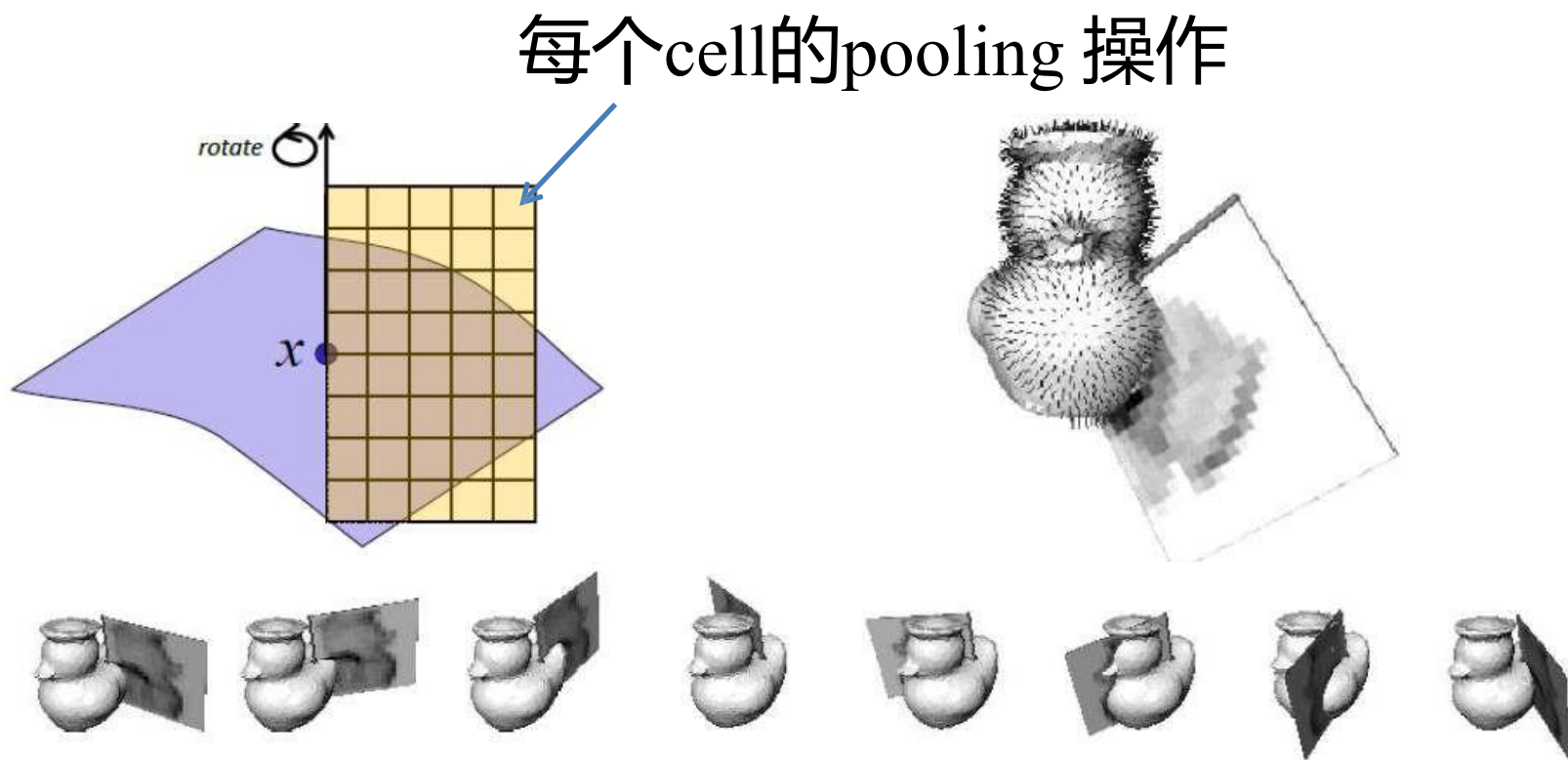
$$S_O: R^3 \rightarrow R^2$$
$$S_O(x) \rightarrow (\alpha, \beta) = (\sqrt{\|x - p\|^2 - (n \cdot (x - p))^2}, n \cdot (x - p))$$

Spin Images



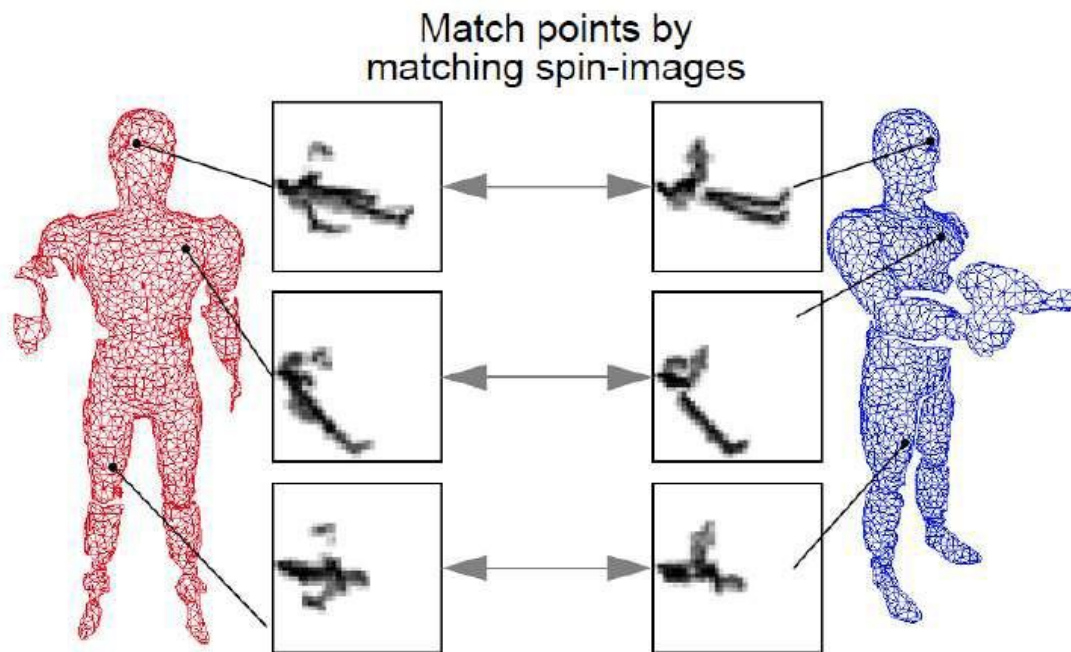
Spin Images

- Spin images的生成过程可以被可视化为一个围绕定向点基旋转的薄片，在扫描空间时积累点。



Spin Images

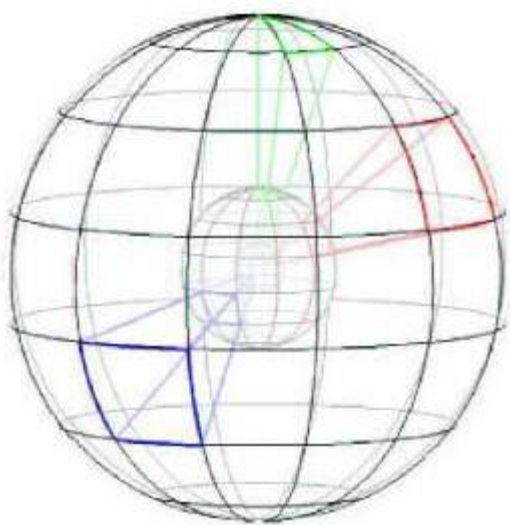
- 匹配特征点(例如, 用3D兴趣点检测器检测)



- Spin image 也可以作为基于支持向量机(SVMs)的分类器学习的特征

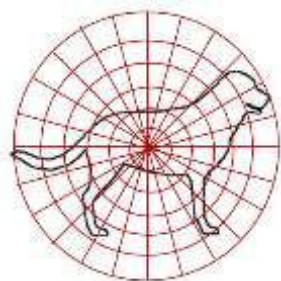
3D Shape Context

[Frome et al, Recognizing Objects in Range Data Using Regional Point Descriptors, ECCV 2004]



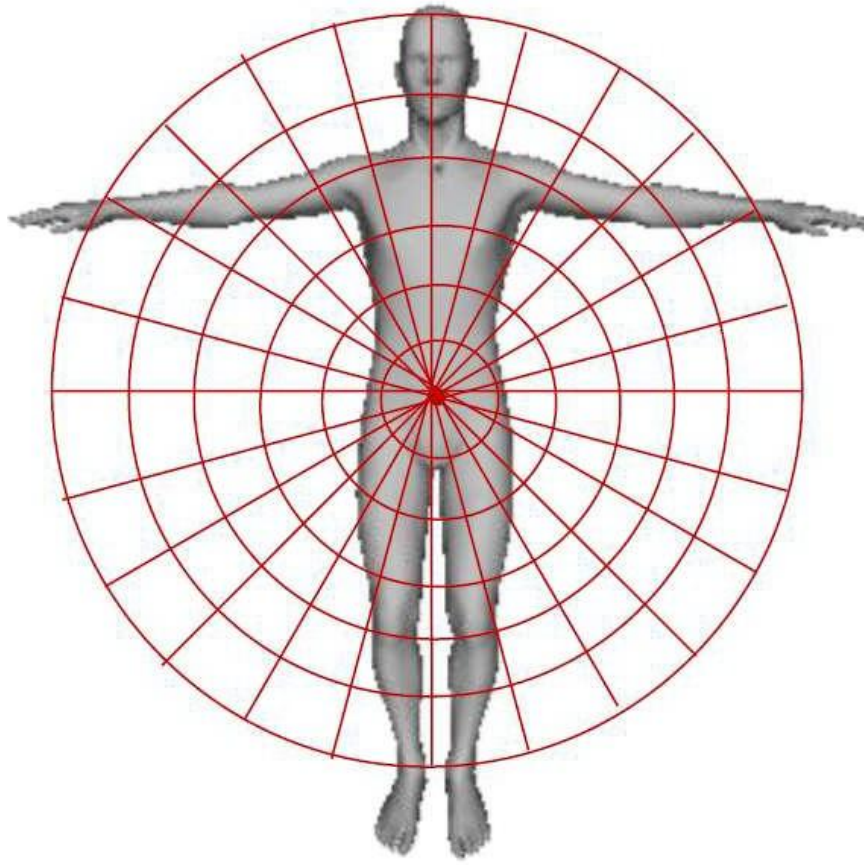
对于物体表面的特定点:

- 将球面北极与该点的表面法线对齐
- 计算球体每个bin中的点的数目，并将其归一化

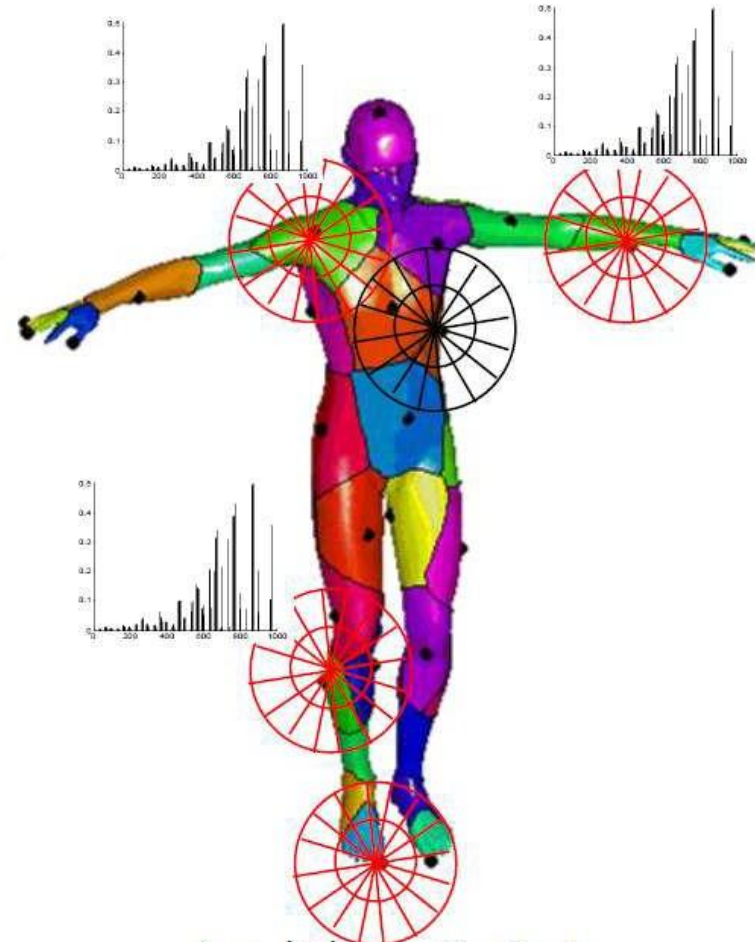


Inspired by 2D Shape Context
[Belongie et al, PAMI 2002]

3D Shape Context



Global Shape Context



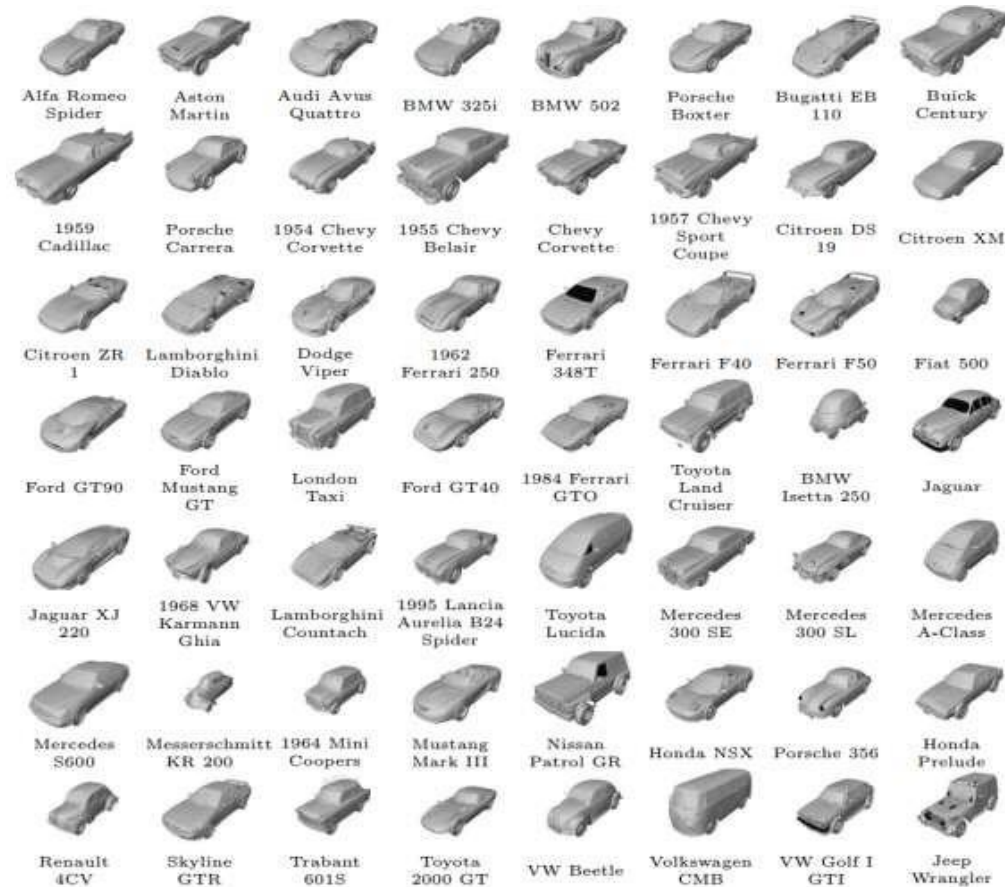
Local Shape Contexts

3D Shape Context

- 三维形状的环境实际上类似于spin image
 - ◆ Spin image使用圆柱体作为支撑区域，其轴与基准点表面法线对齐，而不是球体。
 - ◆ Spin images 也将支持区域分割成单元并计算直方图，但在方位角方向上没有分割 (单元是环形的，形成2D直方图，而不是3D直方图)
 - ◆ Spin images 在方位方向(环)上进行pooling，因此具有旋转不变性。
- 通过投影到球面调和基(或存储直方图的移位版本)，可以使三维形状上下文在方位角方向上具有旋转不变性。 [Frome et al, ECCV 2004]

3D Shape Context

Fine-grained Vehicle Categorization using 3D Shape Contexts [Frome et al, ECCV 2004]



三维模型检索

- 在利用3D几何或结构的同时，3D形状的外观或视图也可用于形状描述，其基本思想是相似的物体在**各个角度上看起来都是相近的**，已有一些相关研究。

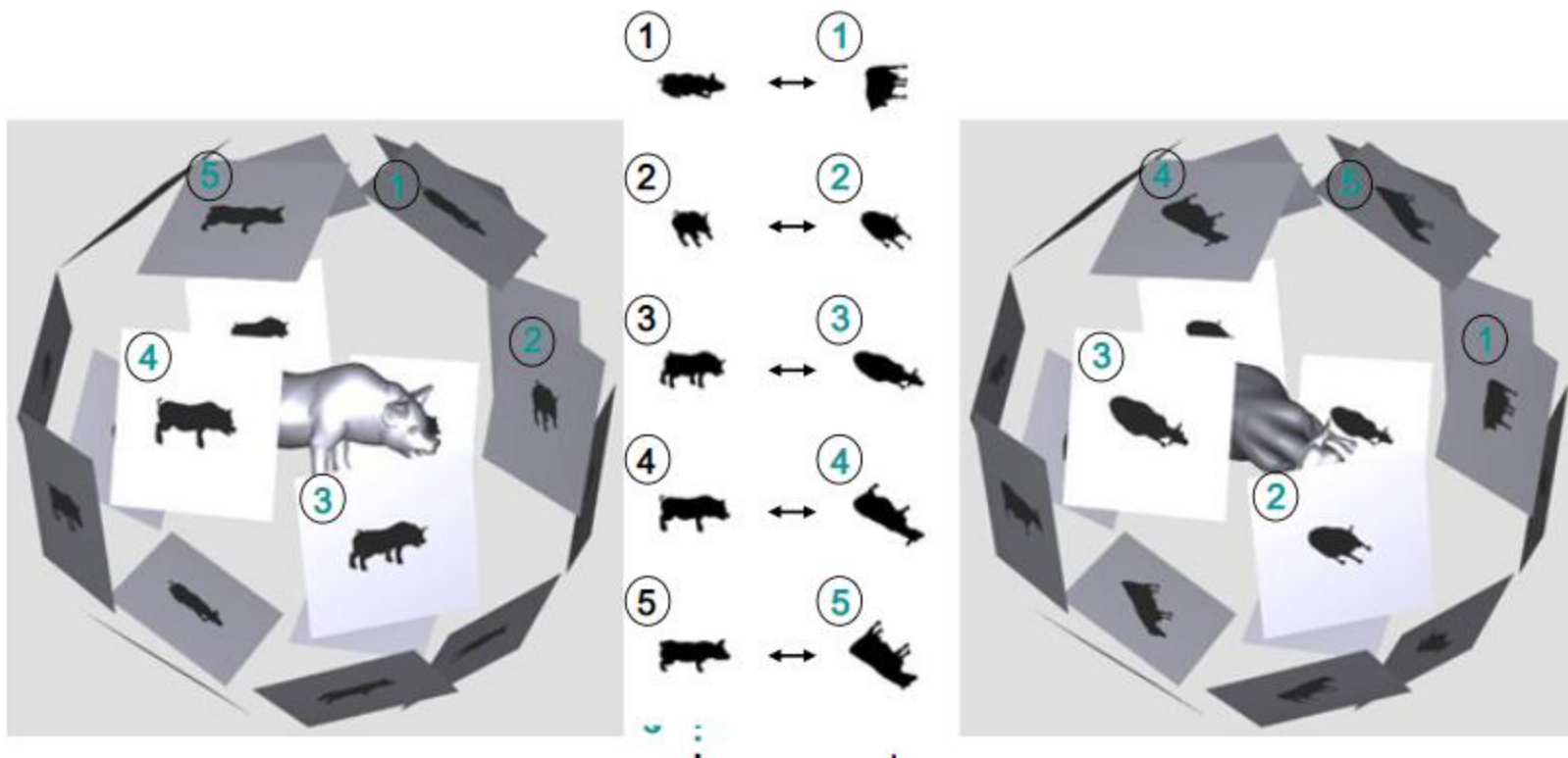


Matching?

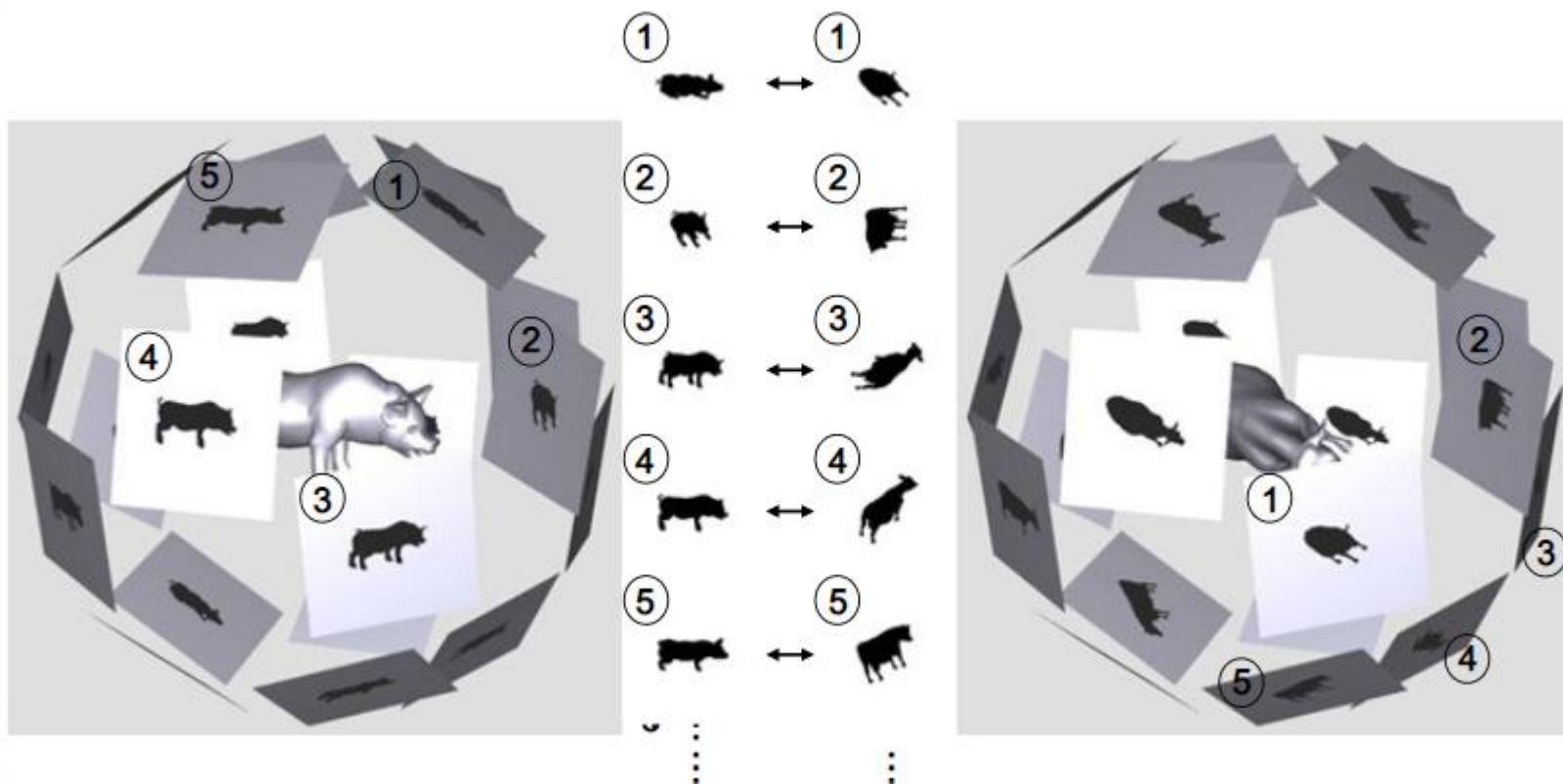


三维模型检索-Light Field

■ 渲染不同角度下的模型图像

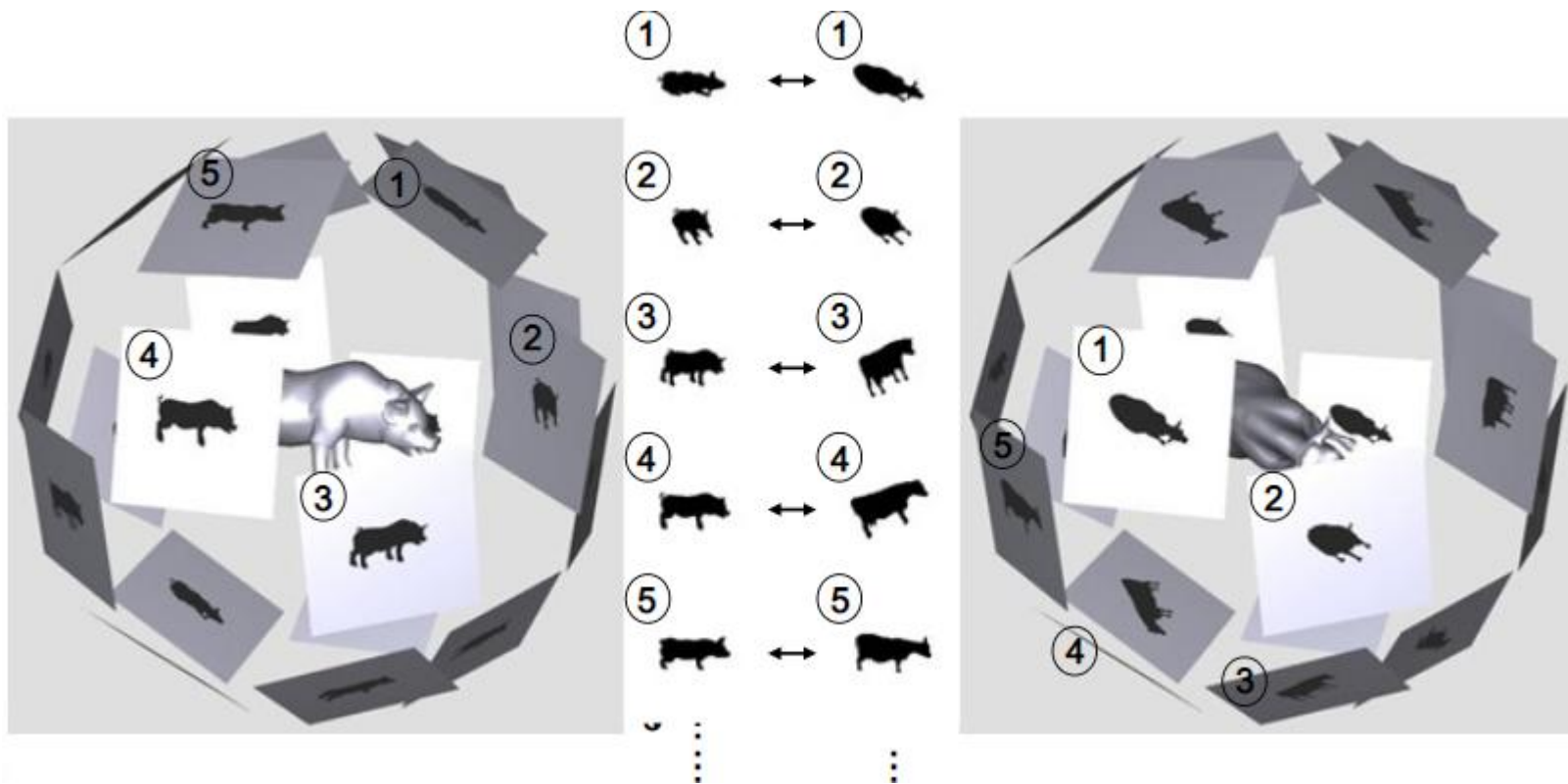


三维模型检索-Light Field



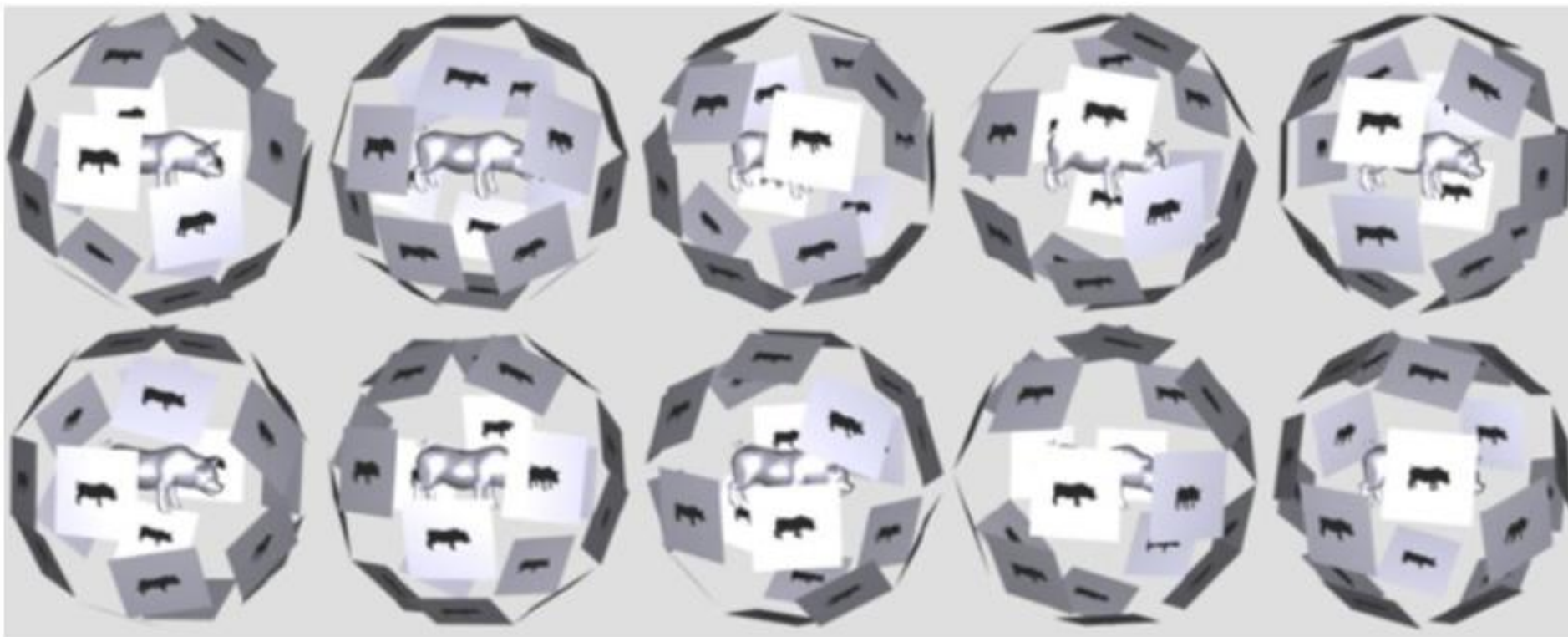
三维模型检索-Light Field

- 最小所有相应二维图像之间的误差



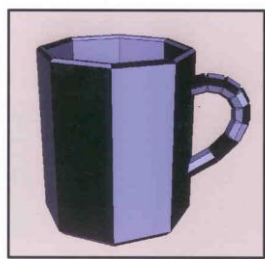
三维模型检索-Light Field

- 找到模型之间的最相似视角的对应



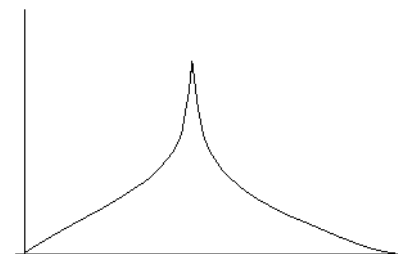
Shape Distributions

- 简洁的形状描述符
- 常见的参数化
- 随机点函数



3D Model

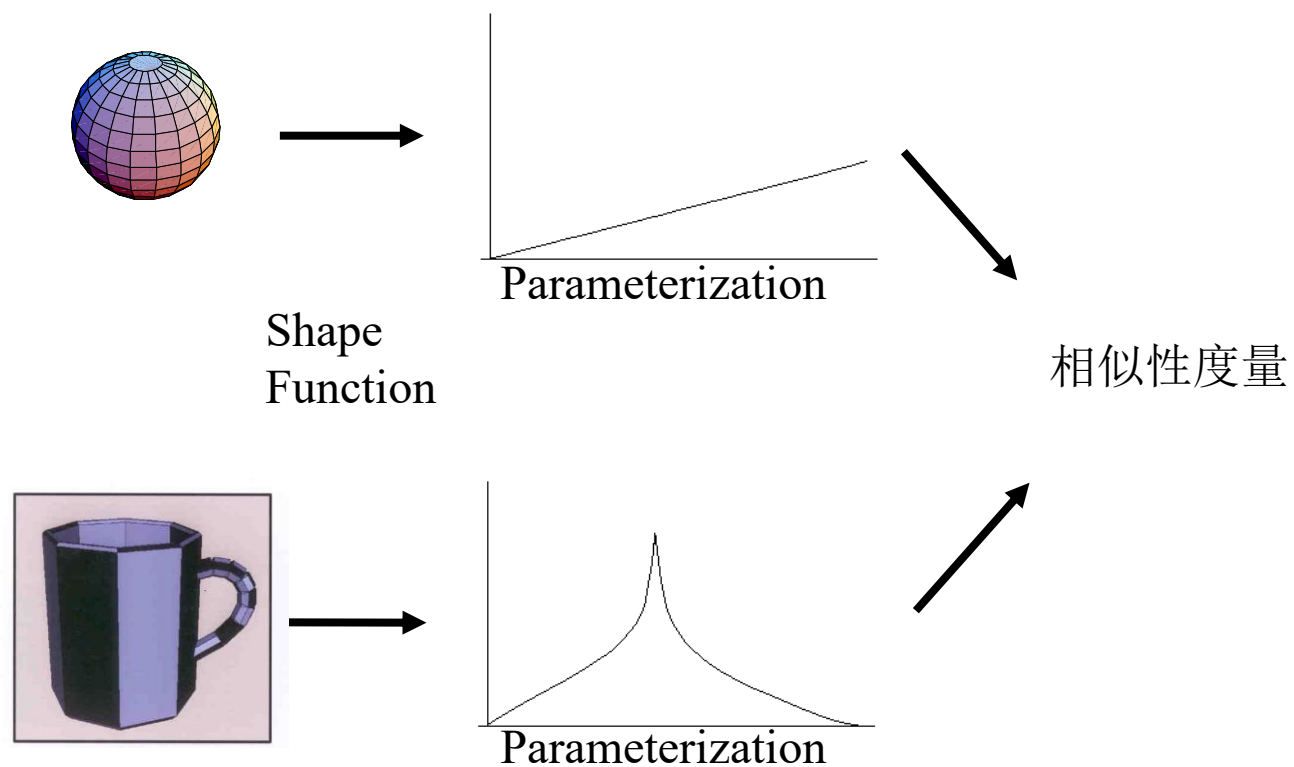
随机采样



Parameterization

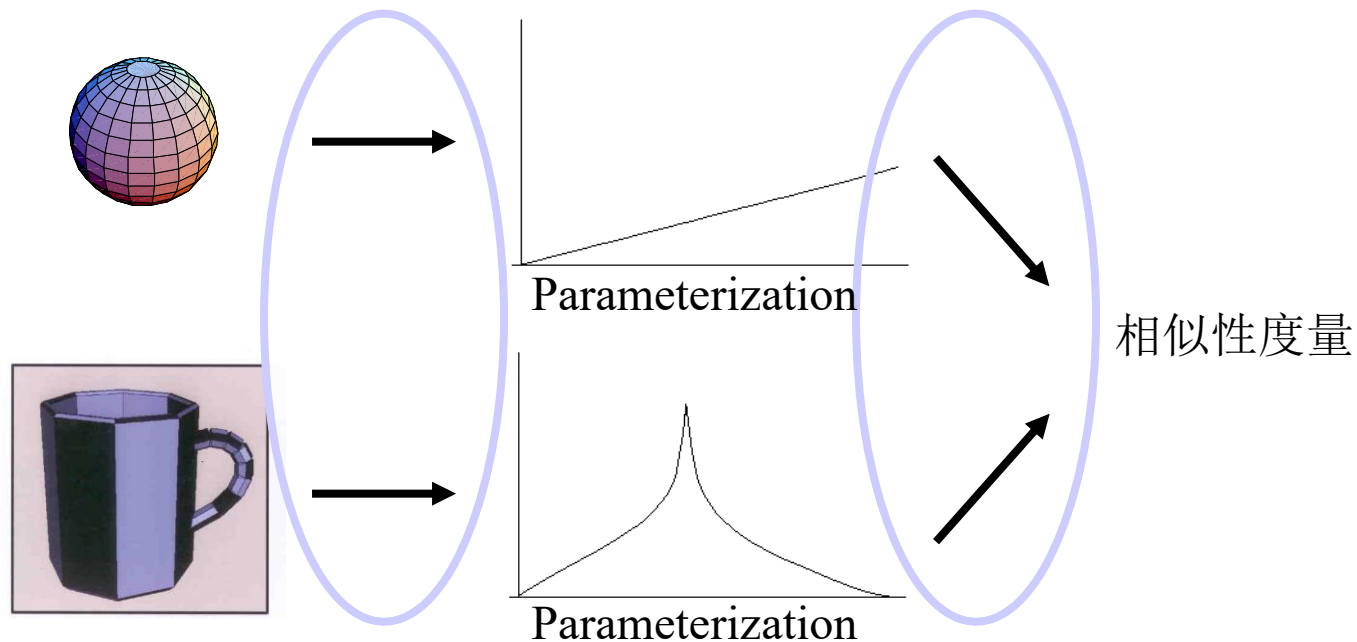
Shape
Distribution

Shape Distributions



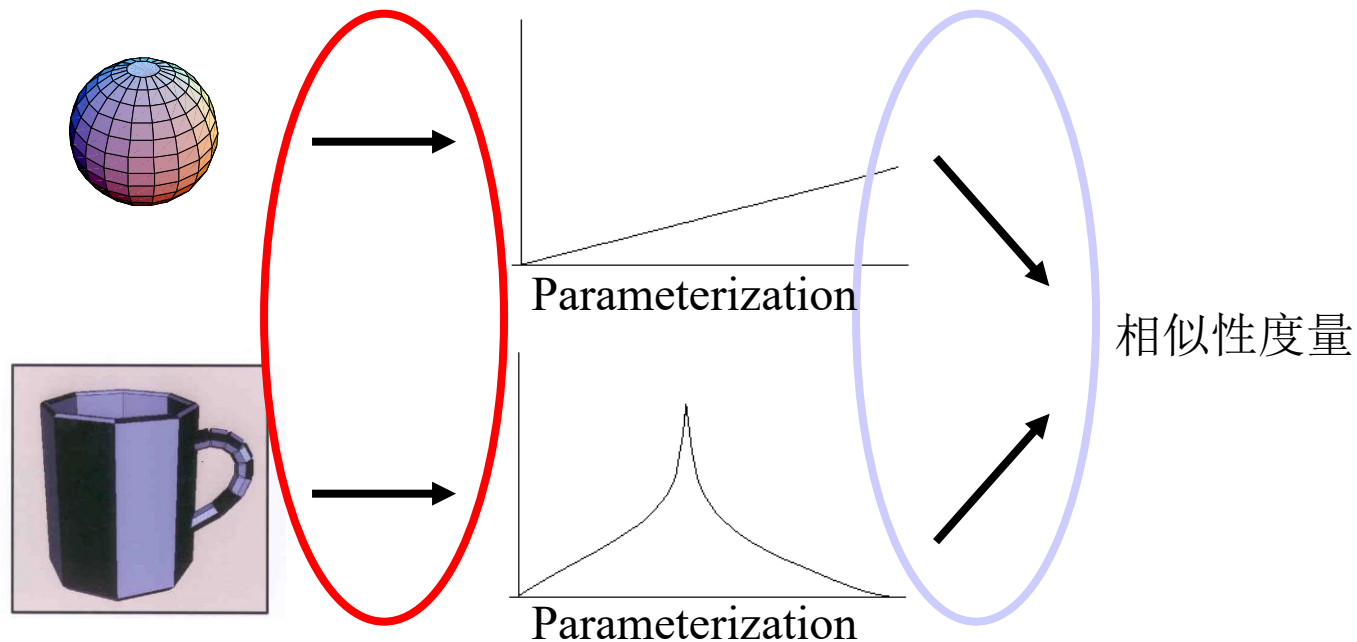
影响因素

- 采用哪种 Shape function?
- 如何比较shape distributions?



影响因素

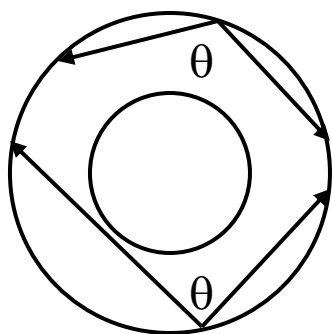
- 采用哪种 Shape function?
- 如何比较shape distributions?



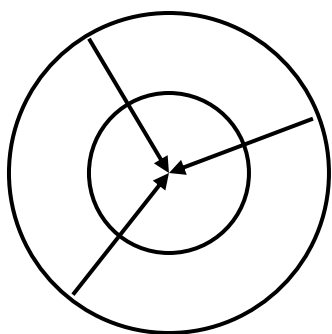
Shape Function

■ 简单的计算方式(~ 1)

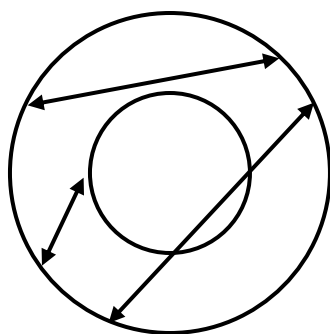
- ◆ 基于随机点
- ◆ 角度, 距离, 面积, 体积



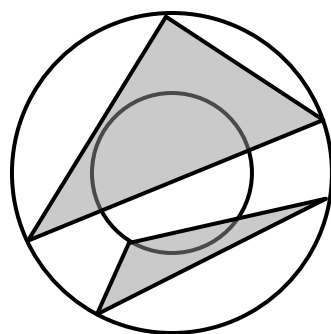
A3



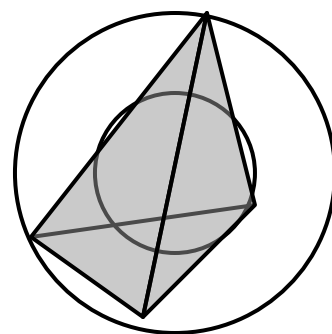
D1



D2



D3

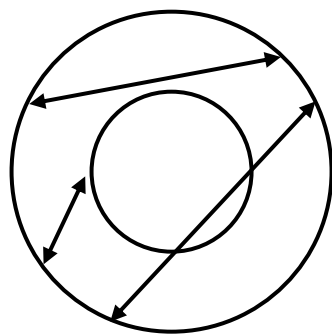


D4

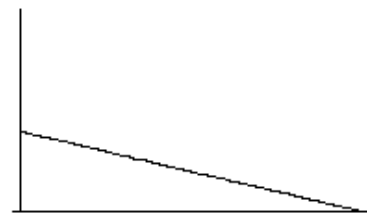
[Ankerst99]

Shape Function

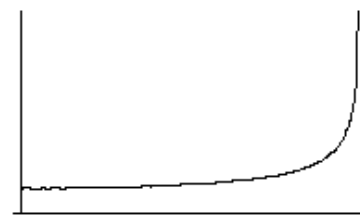
■ 表面上两个随机点之间的距离



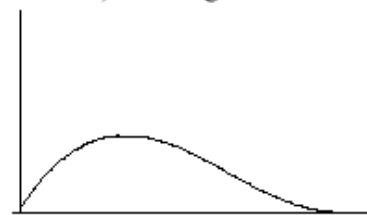
D2



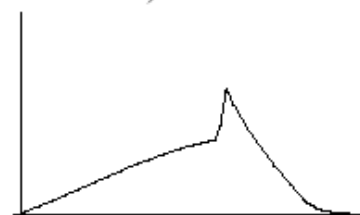
a) Line segment.



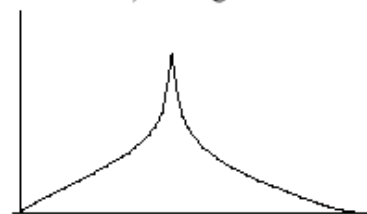
b) Circle.



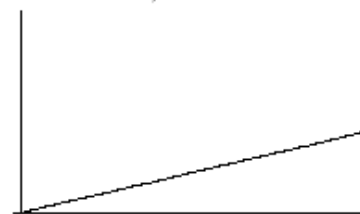
c) Triangle.



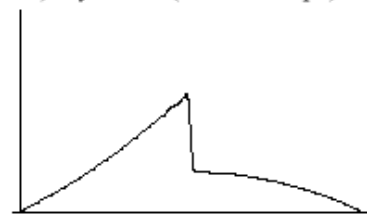
d) Cube.



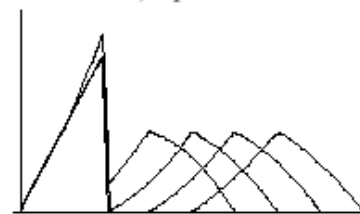
e) Cylinder (without caps).



f) Sphere.



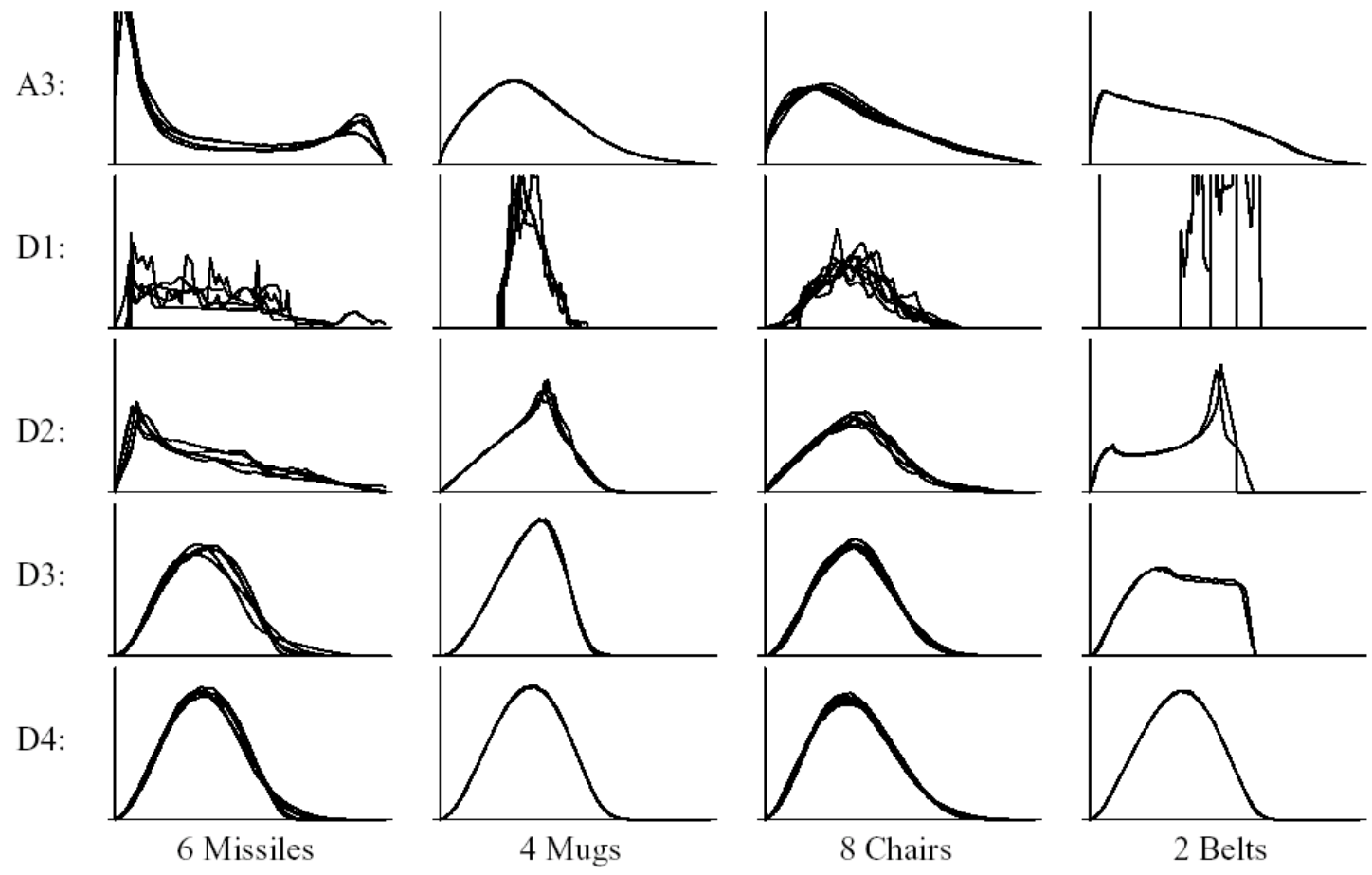
g) Two adjacent unit spheres.



h) Two unit spheres separated by 1, 2, 3, and 4 units.

Shape Function

■ 预览



Shape Function-关键问题

- **是否具有不变性?**

- ◆ 旋转, 平移, 镜像(不是比例)

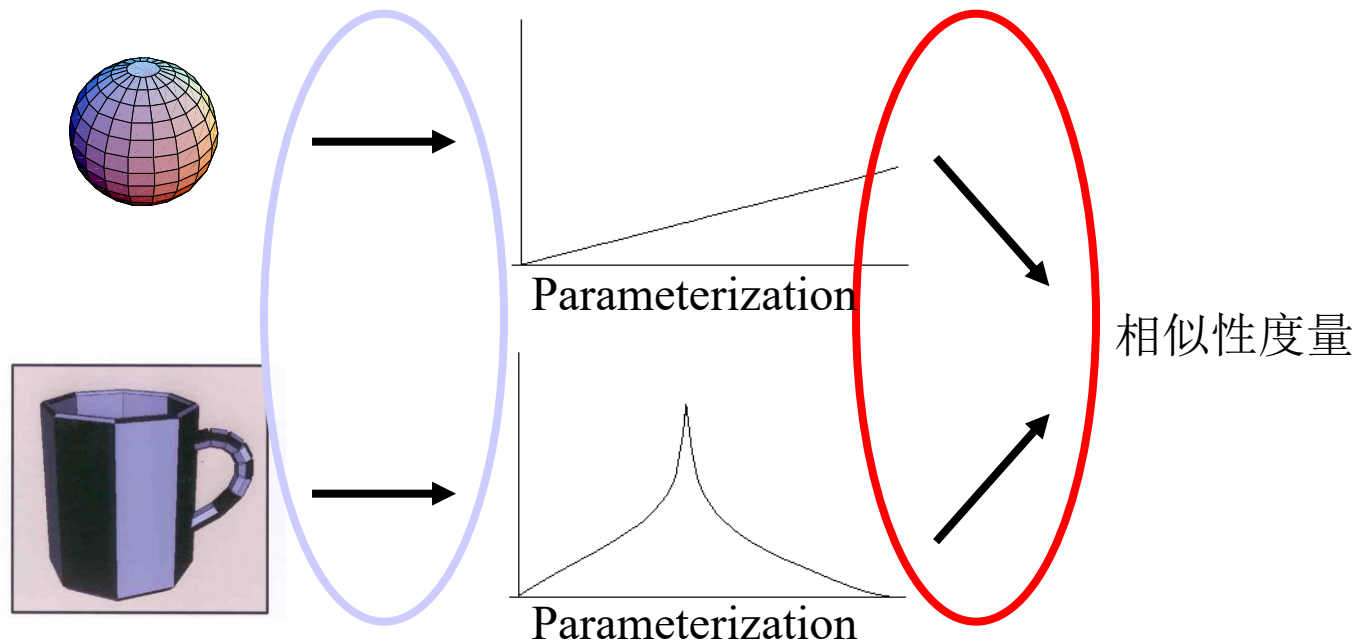
- **是否具有鲁棒性?**

- ◆ 对噪音、裂缝、插入和删除操作是否鲁棒

- **是否具有可描述性?**

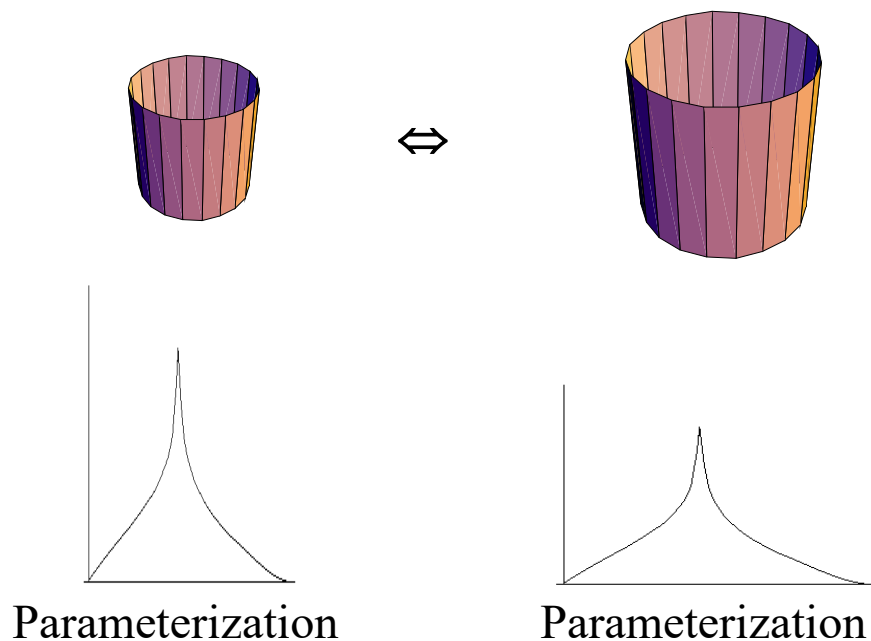
影响因素

- 采用哪种 Shape function?
- 如何比较shape distributions?

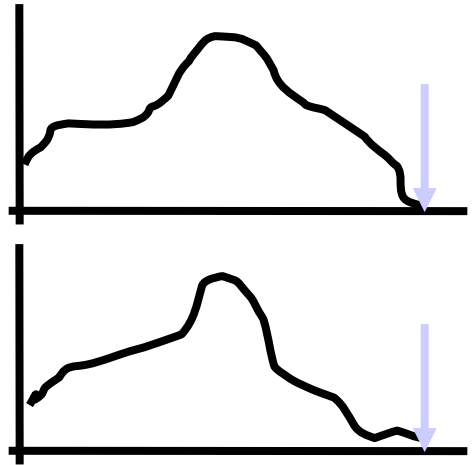


比较

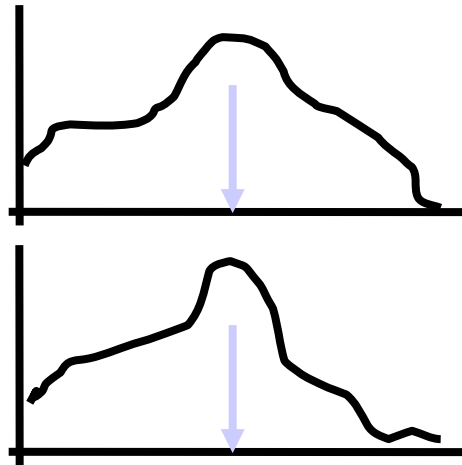
- 规模标准化(Normalize for scale)
- 比较形状分布(Compare shape distributions)



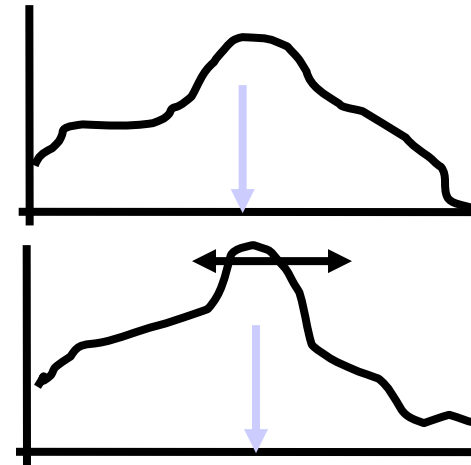
Normalize for scale



max



mean



search

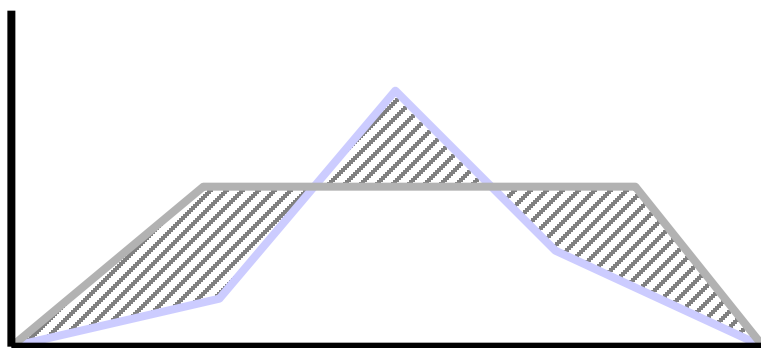
Compare shape distributions

■ 计算简单的选项($\sim .1\text{ms}$)

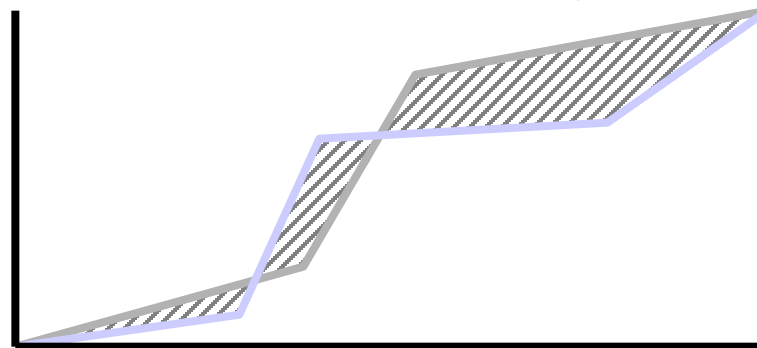
- ◆ Ln norms of densities (PDF) 或
- ◆ cumulative densities (CDF)

■ 更复杂的选项

- ◆ 推土机距离(Earth mover's distance), 巴氏距离(Bhattacharyaa distance)



PDF



CDF₉₈

推土机距离(Earth mover's distance)

- 假设: $P = \{(P_1, w_{p_1}), (P_2, w_{p_2}), \dots, (P_M, w_{p_M})\}$
- $Q = \{(Q_1, w_{q_1}), (Q_2, w_{q_2}), \dots, (Q_N, w_{q_N})\}$, p_i 为P的某个特征, w_{p_i} 为对应的权重, 而 q_j 是Q的某个特征, w_{q_j} 为对应的权重。另外再给一个特征集合P和特征集合Q之间的距离矩阵 d_{ij} , 每一项 d_{ij} 代表 p_i 和 q_j 的距离 (例如L1 L2距离), 希望找到一个流动F, 每一项代表从 p_i 到 q_j 的流动数量, 推土机距离为下面代价函数的最小值:

$$work(P, Q, F) = \sum_{i=1}^M \sum_{j=1}^N d_{ij} f_{ij}$$

谢谢！ 欢迎交流！

高林

gaolin@ict.ac.cn